Multi-Agent Financial Analysis System

AAI-520 Final Team Project, University of San Diego

Group 7: Eric Hernandez, Nolan Robbins, Pranav Dhinakar

GitHub: https://github.com/pdhinaka/AAI-520-NLP-Final-Team-Project-Group7

Team contributions

Member	What they built
Pranav Dhinakar	Data tools, LLM wrapper and cache, prompt chain, final consolidation and integration
Nolan Robbins	Routing, evaluator-optimizer, original notebook Sections 3, 5 and 6
Eric Hernandez	Research agent structure (planner, tool loop, reflection, memory, report fields), later reworked during consolidation

The commit history in the repository shows each contribution in detail.

This notebook builds and evaluates an investment research system for one stock, Apple (AAPL). A research agent plans its research, chooses and calls data tools, critiques and revises its own report, and keeps notes for its next run. Three workflows (prompt chaining, routing and an evaluator-optimizer) run beside it on the same data tools.

Section 2 shows how the parts fit together. Sections 3 to 6 run the data tools and the three workflows, Section 7 demonstrates the agent's four functions, Section 8 runs everything end to end, and Section 9 covers evaluation and what we changed along the way. Each result is followed by a short interpretation of that output.

All outputs come from one live run on 2026-10-18 with Claude Haiku 4.5. Every API and LLM reply was cached on disk, and this copy was executed with `CACHE_MODE=offline`, which replays those replies with no network calls.

1. Setup

[1]:
```
import json
import logging
import os
import re
import subprocess
import sys
import textwrap

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from IPython.display import Markdown, display
from matplotlib.patches import FancyArrowPatch, FancyBboxPatch

from finagent import config, pipeline
from finagent.agent import memory, reflection, research_agent
from finagent.workflows import evaluator_optimizer as eo
```

```
from finagent.workflows import prompt_chain, routing

TICKER = "AAPL"
THRESHOLD = 0.8  # evaluator-optimizer pass threshold, used everywhere

# The news and FRED tools build date windows from "today", and those
# dates are part of their cache keys. Pinning today to the live-run
# date lets an offline replay on a later day hit the same entries.
config.AS_OF_DATE = os.getenv("AS_OF_DATE", "2026-10-18")
# Runs are saved to data/runs/ only when live, so the cost cells can
# read the live numbers back during an offline replay.
LIVE = config.CACHE_MODE != "offline"
print(f"cache mode {config.CACHE_MODE}, as of {config.AS_OF_DATE}, "
      f"model {config.LLM_MODEL}")

logging.basicConfig(level=logging.WARNING)
for name in ("httpx", "yfinance", "urllib3", "anthropic", "finagent"):
    logging.getLogger(name).setLevel(logging.ERROR)
pd.set_option("display.max_colwidth", 90)
pd.set_option("display.width", 120)

# Chart style: categorical slots from a colorblind-checked palette, muted
# ink for axes and reference lines, light grid.
SERIES = ["#2a78d6", "#eb6834", "#1baf7a"]
INK, MUTED = "#0b0b0b", "#898781"


def style(ax, grid_axis="y"):
    """Recessive axes: no top/right spines, light grid behind data."""
    for side in ("top", "right"):
        ax.spines[side].set_visible(False)
    for side in ("left", "bottom"):
        ax.spines[side].set_color(MUTED)
    ax.tick_params(colors=MUTED, labelcolor=INK)
    ax.grid(axis=grid_axis, color="#e6e5e0", lw=0.8)
    ax.set_axisbelow(True)
    ax.title.set_ha("left")
    ax.title.set_position((0, 1.02))


def wrap(text, width=100, indent=""):
    """Wrap long model text so it fits the PDF page."""
    return textwrap.fill(str(text), width, initial_indent=indent,
                         subsequent_indent=" " * len(indent))


def md(text):
    """Model text as Markdown, with $ escaped so "$94.9B ... $3" is not
    read as LaTeX math."""
    return Markdown(str(text).replace("$", r"\$"))


def live_runs(prefix, n=1):
    """The n most recent saved runs that made live LLM calls, oldest
    first. An offline replay (or a run that hit a warm cache) reports
    zero usage, so the cost cells read the live numbers from here."""
    runs = []
    for path in sorted((config.DATA_DIR / "runs").glob(
            f"{prefix}_{TICKER}_*.json")):
        run = json.loads(path.read_text())
        if run.get("usage", {}).get("calls"):
```

```
            runs.append(run)
    return runs[-n:]
```

```
cache mode offline, as of 2026-10-18, model claude-haiku-4-5
```

2. Agent Design and Workflows

[2]:
```
# Architecture: five data sources behind one tool registry; the agent
# (left) and the three workflows (right) both draw on it. Arrows show
# data flow; the dashed loop is memory carried to the next run.
fig, ax = plt.subplots(figsize=(12, 7.2))
ax.set(xlim=(0, 100), ylim=(-5, 60))
ax.axis("off")
PANEL, BOX = "#f4f3ef", "#ffffff"


def box(x, y, w, h, text, fc=BOX, ec=MUTED, size=8.5, weight="normal"):
    ax.add_patch(FancyBboxPatch(
        (x, y), w, h, boxstyle="round,pad=0.3,rounding_size=0.8",
        fc=fc, ec=ec, lw=1))
    ax.text(x + w / 2, y + h / 2, text, ha="center", va="center",
            fontsize=size, color=INK, fontweight=weight)


def arrow(p, q, rad=0.0, ls="-", color=MUTED, label=None, at=0.5,
          dx=0, dy=0):
    ax.add_patch(FancyArrowPatch(
        p, q, arrowstyle="-|>", mutation_scale=11, color=color, lw=1.1,
        linestyle=ls, connectionstyle=f"arc3,rad={rad}",
        shrinkA=2, shrinkB=2))
    if label:
        x = p[0] + (q[0] - p[0]) * at + dx
        y = p[1] + (q[1] - p[1]) * at + dy
        ax.text(x, y, label, fontsize=7.5, color=MUTED, ha="center",
                va="center", backgroundcolor="white")


# Data sources
sources = ["Yahoo Finance\nprices, profile,\nfinancials",
           "NewsAPI +\nYahoo search\nnews", "FRED\nmacro series",
           "SEC EDGAR\nfilings", "Alpha Vantage\nEPS vs\nestimates"]
for i, s in enumerate(sources):
    x = 2 + i * 11.6
    box(x, 50, 10.4, 8, s, fc=PANEL, size=7.5)
    arrow((x + 5.2, 50), (x + 5.2, 44.6))

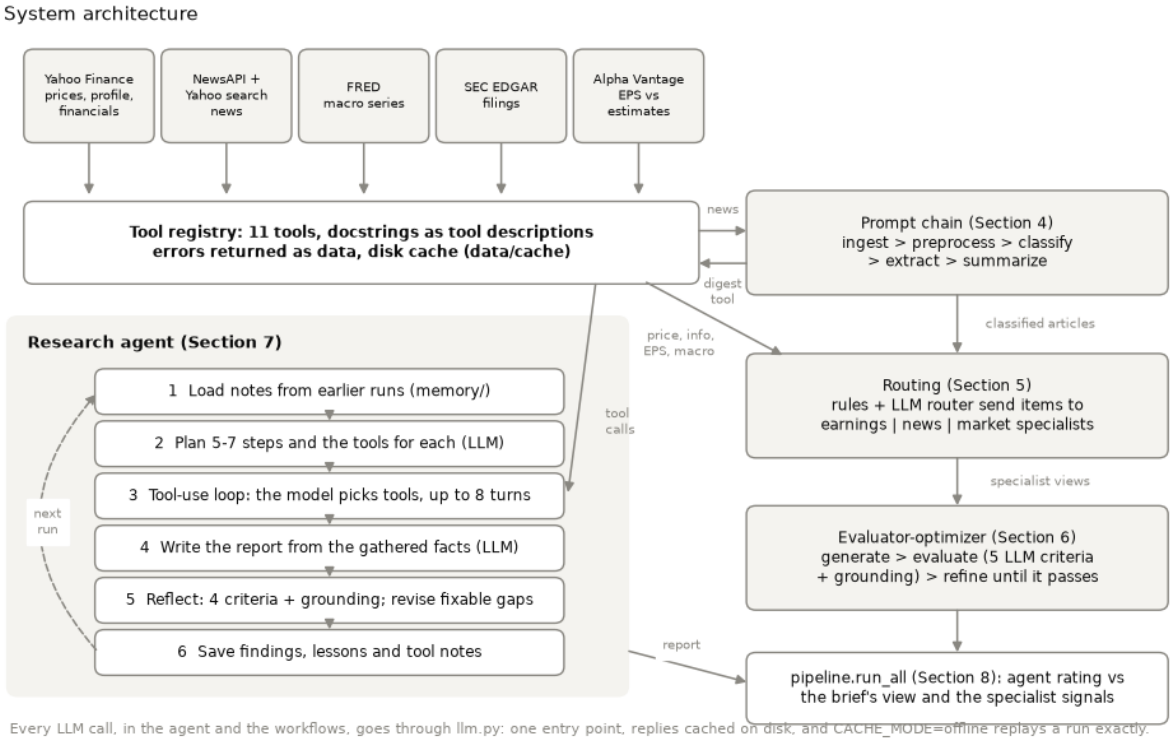
box(2, 37, 56.4, 7, "Tool registry: 11 tools, docstrings as tool "
    "descriptions\nerrors returned as data, disk cache (data/cache)",
    size=8.5, weight="bold")

# Research agent
ax.add_patch(FancyBboxPatch((0.5, -1), 52, 34.5,
                            boxstyle="round,pad=0.3,rounding_size=1",
                            fc=PANEL, ec="none"))
ax.text(2, 32, "Research agent (Section 7)", fontsize=9.5,
        fontweight="bold", color=INK, va="top")
steps = ["1  Load notes from earlier runs (memory/)",
         "2  Plan 5-7 steps and the tools for each (LLM)",
         "3  Tool-use loop: the model picks tools, up to 8 turns",
         "4  Write the report from the gathered facts (LLM)",
```

```
        "5  Reflect: 4 criteria + grounding; revise fixable gaps",
        "6  Save findings, lessons and tool notes"]
ys = [25, 20.2, 15.4, 10.6, 5.8, 1]
for s, y in zip(steps, ys):
    box(8, y, 39, 3.6, s)
for a, b in zip(ys, ys[1:]):
    arrow((27.5, a), (27.5, b + 3.6))
arrow((8, ys[-1] + 1.8), (8, ys[0] + 1.8), rad=-0.45, ls="--",
      label="next\nrun", at=0.5, dx=-4.3)
arrow((50, 37), (47.6, ys[2] + 1.8))
ax.text(50.8, 24, "tool\ncalls", fontsize=7.5, color=MUTED, ha="left",
        va="center")

# Workflows
box(63, 36, 35, 9, "Prompt chain (Section 4)\ningest > preprocess > "
    "classify\n> extract > summarize", fc=PANEL)
arrow((58.4, 41.6), (63, 41.6))
arrow((63, 38.6), (58.4, 38.6))
ax.text(60.7, 43.2, "news", fontsize=7.5, color=MUTED, ha="center")
ax.text(60.7, 37.4, "digest\ntool", fontsize=7, color=MUTED,
        ha="center", va="top")
box(63, 21, 35, 9, "Routing (Section 5)\nrules + LLM router send items "
    "to\nearnings | news | market specialists", fc=PANEL)
box(63, 7, 35, 9, "Evaluator-optimizer (Section 6)\ngenerate > evaluate "
    "(5 LLM criteria\n+ grounding) > refine until it passes", fc=PANEL)
arrow((80.5, 36), (80.5, 30), label="classified articles", dx=7)
arrow((54, 37), (66, 30))
ax.text(60, 31.2, "price, info,\nEPS, macro", fontsize=7.5,
        color=MUTED, ha="right", va="center")
arrow((80.5, 21), (80.5, 16), label="specialist views", dx=7)
box(63, -3.5, 35, 6, "pipeline.run_all (Section 8): agent rating "
    "vs\nthe brief's view and the specialist signals")
arrow((80.5, 7), (80.5, 2.5))
arrow((52.5, 3), (63, 0), label="report", at=0.45, dy=1.8)

ax.text(0.5, -5, "Every LLM call, in the agent and the workflows, goes "
        "through llm.py: one entry point, replies cached on disk, and "
        "CACHE_MODE=offline replays a run exactly.", fontsize=8,
        color=MUTED, va="bottom")
ax.set_title("System architecture", loc="left", fontsize=11, color=INK)
plt.show()
```



We built the system in plain Python rather than on an agent framework such as LangGraph. Each agent function and each workflow is a short module of ordinary functions, so the control flow in the diagram is the control flow in the code: `research_agent.run` reads top to bottom as load notes, plan, tool loop, write, reflect and revise, save notes. A framework would have added a dependency and a layer of abstraction without giving us anything the four functions or three patterns need.

Every LLM call goes through one module, `llm.py`. It holds the client, the model name and the token accounting, and it is where caching happens: a reply is stored on disk under a hash of the full request (model, system prompt, messages and tools), so the same request always returns the same reply. With `CACHE_MODE=offline` a whole run replays with no network access. This is what makes the notebook reproducible, because the installed Anthropic SDK (1.9.0) does not accept a temperature parameter and sampling alone would give different text on every run. The data tools are cached the same way, and the date windows of the news and FRED tools are pinned to `AS_OF_DATE` so a replay on a later day still finds the same entries.

Tools never raise an exception into the agent. `registry.call` catches it and returns `{"error": ...}`, which goes back to the model as a tool result marked as an error, so the model can choose another tool (Section 7.2 shows this on purpose). Routing treats a failed tool the same way: it skips the tool and lists it.

Finally, we keep arithmetic out of the model. Returns, growth rates, beat rates, the position in the 52-week range and the upside to the analyst target are computed in Python from the tool results, and the model is asked to interpret them. Grounding checks in the evaluator-optimizer and in the agent's self-reflection then look up every number the model wrote in those facts. The model's job is judgment: which numbers matter and what they imply for a Buy, Hold or Sell view.

3. Data Tools

Every data source is wrapped as a plain Python function that returns JSON-friendly dicts and is cached on disk. The registry exposes them to the LLM as tools, with each function's docstring used as the tool description.

[3]:
```
from finagent.tools import market_data, registry

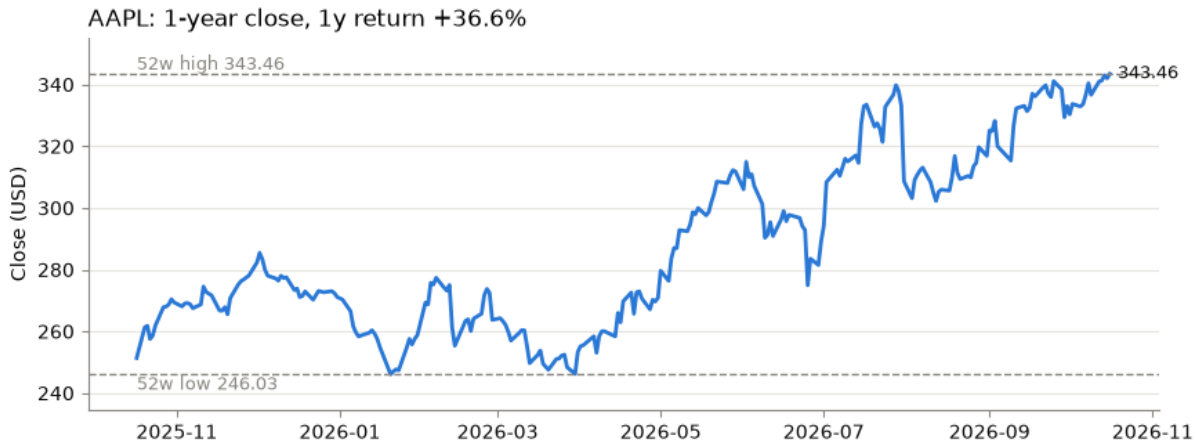
print(registry.describe())
```

```
- get_company_info: Company profile and key valuation metrics.
- get_price_summary: Recent stock performance at a glance.
- get_price_history: Daily (or other interval) OHLCV price history for a stock.
- get_financials: Key lines from the income statement, balance sheet, and cash flow.
- get_news: Recent news articles about a company.
- get_news_digest: Short digest of recent news for a stock, built by the prompt chain.
- get_macro_snapshot: Current macroeconomic backdrop in one call.
- get_series: Observations for one FRED economic data series.
- get_recent_filings: Recent SEC filings for a company, newest first.
- get_filing_text: Plain text of an SEC filing document (retrieval tool).
- get_earnings: Recent quarterly earnings: reported EPS vs. analyst estimate.
```

[4]:
```
# One-year closing price with the 52-week high and low, the context
# the market specialist reasons about in section 5.
prices = pd.DataFrame(market_data.get_price_history(TICKER, period="1y"))
prices["date"] = pd.to_datetime(prices["date"])
summary = market_data.get_price_summary(TICKER)

fig, ax = plt.subplots(figsize=(10, 3.5))
ax.plot(prices["date"], prices["close"], color=SERIES[0], lw=2)
# High label sits above its line and low label below, so neither can
# overlap the price line.
for key, label, va in (("high_52w", "52w high", "bottom"),
                       ("low_52w", "52w low", "top")):
    ax.axhline(summary[key], color=MUTED, ls="--", lw=1)
    ax.text(prices["date"].iloc[0], summary[key],
            f"{label} {summary[key]:.2f}", va=va, color=MUTED,
            fontsize=9)
ax.annotate(f"{summary['last_close']:.2f}",
            (prices["date"].iloc[-1], summary["last_close"]),
            textcoords="offset points", xytext=(4, 0), va="center",
            color=INK, fontsize=9)
ax.margins(y=0.12)
ret_1y = summary.get("return_1y")
ret_text = f"{ret_1y:+.1%}" if ret_1y is not None else "n/a"

ax.set(
    ylabel="Close (USD)",
    title=f"{TICKER}: 1-year close, 1y return {ret_text}"
)
style(ax)
plt.show()
```



4. Workflow 1: Prompt Chaining

The chain turns raw news into a digest the rest of the system can rely on. Ingest and preprocess are plain Python because they are mechanical: fetching from NewsAPI and Yahoo, removing duplicates, dropping articles that never name the company, and capping each outlet at four articles so one source cannot dominate. Doing this in code makes every filter decision exact, free and explainable (4.2 lists each dropped article with its reason). Classify, extract and summarize need language understanding, so they go to the LLM.

The sentiment and topic counts in the digest are computed in Python from the classify labels rather than written by the model, so they always agree with the table in 4.3. The summary cites articles by id, and a check lists any citation that does not match a kept article. All LLM replies are cached, so a replay produces the same labels and the same digest.

4.1 Run the chain

[5]:
```
chain = prompt_chain.run_chain(TICKER, save=LIVE)
pre = chain["steps"]["preprocess"]
print(
    f"{chain['model']}: ingested {len(chain['steps']['ingest'])} "
    f"articles, kept {len(pre['articles'])} after preprocessing"
)
```

```
claude-haiku-4-5: ingested 30 articles, kept 18 after preprocessing
```

4.2 Preprocess: what was dropped and why

[6]:
```
pd.DataFrame(pre["dropped"])
```

[6]:

	title	reason
0	What Does Broadcom (AVGO) Face As AI Chip Supply Turns Into A Bottleneck?	does not mention the company
1	Spy Games in Silicon Valley	does not mention the company
2	Taiwan Semiconductor's (TSM) Profit Rose 69% Last Quarter. Is the Stock Still Cheap?	does not mention the company
3	iPhone Duo Preorders Sell Out Rapidly As Delivery Estimates Slip Into January	does not mention the company
4	What to Expect in Markets This Week: Meta, Tesla Report; Airlines, Streaming Giants' R...	does not mention the company
5	A week with RTX Spark: everything we've learned using Nvidia's new laptops	does not mention the company

	title	reason
6	The iPhone Duo is now available to preorder at $1,999	over 4 from The Verge
7	Meta and Apple team up to make Muse AI models run better on Macs	over 4 from The Verge
8	The best feature of Apple's HomeView might be its simplicity	over 4 from The Verge
9	Which Apple Home Devices Get Siri AI, and When	over 4 from MacRumors
10	Apple Releases New AirPods Beta Firmware	over 4 from MacRumors
11	Apple Releases Fourth iOS 27.2 Beta	over 4 from MacRumors

Preprocessing kept 18 of the 30 ingested articles. Six were cut by the four-per-outlet cap (three each from The Verge and MacRumors). The other six were dropped for not naming the company, and five of those are about other subjects: Broadcom, TSMC, Nvidia laptops, a weekly market preview and a piece titled "Spy Games in Silicon Valley". The sixth is a cost of the rule. "iPhone Duo Preorders Sell Out Rapidly As Delivery Estimates Slip Into January" came from Yahoo with an empty text field, and its title names the product but not Apple, so it was dropped although it is relevant. The same story still reached the digest through two other articles (9 and 17 in 4.5).

4.3 Classify: topic, sentiment, relevance

[7]:
```
pd.DataFrame([
    {"id": a["id"], "title": a["title"][:60], **a["labels"]}
    for a in chain["steps"]["classify"]
])
```

[7]:

	id	title	topic	sentiment	relevant	reason
0	1	The $5 Trillion Club: Which Is the Better Stock to Own, Appl	analyst_rating	neutral	True	Analyst comparison of AAPL vs NVDA provides investment perspective for shareholders.
1	2	Apple Keeps Hitting Record Highs. Bitcoin Hasn't Set One All	analyst_rating	positive	True	Article highlights AAPL reaching record highs, suggesting positive investor sentiment.
2	3	Own VGT? Here's How Much of It Is Really Nvidia, Apple, Micr	other	neutral	False	ETF composition analysis is tangential to AAPL company fundamentals.
3	4	Apple's Market Value Grew From $350 Billion to $5 Trillion D	other	positive	True	Leadership transition and historical market value growth milestone relevant to future ...
4	5	Apple bets on the same Siri AI technology with new HomeView	product	negative	True	New product relies on technology previously associated with a $250M settlement, raisin...
5	6	Apple's new CEO just teamed up with Tim Cook's old rival	other	neutral	True	New CEO's business decisions and partnerships are relevant to shareholder interests.
6	7	$10,000 Invested in Apple Could Be Worth This Much by 2031	analyst_rating	positive	True	Analyst projection of future stock value growth is relevant to shareholder expectations.
7	8	Apple, Nvidia each gave $25M to Trump's White House ballroom	macro	neutral	False	Political donation disclosure is tangential to AAPL business operations and shareholde...
8	9	Apple's $1,999 Foldable iPhone Sold Out in Minutes. Deliveri	product	positive	True	Strong product demand with sellout demonstrates market interest in new foldable iPhone.
9	10	Apple Hit a Record Close Ahead of Its November 2 Report. Her	product	neutral	True	Upcoming earnings report and potential new revenue model from AI features relevant to ...

	id	title	topic	sentiment	relevant	reason
10	11	All the news about the iPhone Duo	product	positive	True	Foldable iPhone launch represents significant new product category for revenue growth.
11	12	Apple TV 4K and HomePod Mini Price Increases 'Much More Stri	product	negative	True	Price increases without corresponding feature improvements may concern investors about...
12	13	Apple's HomeView is just the latest threat to online recipe	product	neutral	False	Third-party impact analysis of HomeView features does not directly affect shareholder ...
13	14	Apple's HomeView has an app problem	product	negative	True	App ecosystem limitations for new HomeView device may hinder adoption and competitive ...
14	15	Apple Pre-Orders Kick Off With First Discounts on HomeView a	product	neutral	False	Retail discount listings on third-party platforms are not core company news for shareh...
15	16	Developers Question Scope of Potential Apple Antitrust Settl	legal	negative	True	Antitrust settlement discussions represent regulatory risk to AAPL's iOS control and b...
16	17	iPhone Duo orders are already delayed into 2027	product	positive	True	Rapid demand-driven order delays indicate strong product demand for iPhone Duo.
17	18	Rare Apple-1 With Steve Jobs Letter Sells for $1.15 Million	other	neutral	False	Vintage Apple-1 auction is historical memorabilia news unrelated to current business o...

The classifier marked 13 of the 18 articles relevant. The five it rejected are tangential to Apple's business (an ETF's holdings, a political donation, recipe websites, retailer discounts and an auction of an Apple-1), and its one-line reasons say so. Only relevant articles go on to extraction, which is why 4.4 has 13 rows.

4.4 Extract: key facts per article

[8]:
```
pd.DataFrame(chain["steps"]["extract"])[
    ["id", "event", "figures", "companies", "dates"]
]
```

[8]:

	id	event	figures	companies	dates
0	1	Motley Fool compares Apple and Nvidia as members of the $5 Trillion valuation club to ...	[{'value': '$5 Trillion', 'context': 'market valuation club'}]	[Apple, Nvidia]	[2026-10-18]
1	2	24/7 Wall St. compares Apple's record highs against Bitcoin's lack of new highs this y...	[]	[Apple]	[2026-10-18]
2	4	Apple's market value grew from $350 billion to $5 trillion during Tim Cook's 15 years ...	[{'value': '$350 Billion', 'context': 'Apple's market value at start of Tim Cook's ten...	[Apple]	[15 Years, 2026-10-17]
3	5	Apple launches a new HomeView device using Siri AI technology, the same technology inv...	[{'value': '$250M', 'context': 'settlement for iPhone users'}]	[Apple]	[2026-10-17]
4	6	Apple's new CEO teams up with Tim Cook's former rival according to TheStreet.	[]	[Apple]	[2026-10-17]
5	7	Motley Fool projects what a $10, 000 investment in Apple could be worth by 2031.	[{'value': '$10,000', 'context': 'investment amount'}]	[Apple]	[2031]

	id	event	figures	companies	dates
6	9	Apple's $1,999 Foldable iPhone sold out in minutes with deliveries now slipping to Jan...	[{'value': '$1,999', 'context': 'Foldable iPhone price'}]	[Apple]	[January, 2026-10-17]
7	10	Apple hit a record close ahead of its November 2 earnings report, with analysis on wha...	[]	[Apple]	[November 2, 2026-10-17]
8	11	Apple announced its foldable iPhone Duo with a 5.4-inch outer display and 7.6-inch inn...	[{'value': '5.4-inch', 'context': 'outer display'}, {'value': '7.6-inch', 'context': '...	[Apple]	[2026-10-16]
9	12	John Gruber concluded that price increases for the new Apple TV 4K and HomePod mini 2 ...	[]	[Apple]	[2026-10-16]
10	14	Apple's smart home device HomeView faced questions about app availability following th...	[]	[Apple]	[2026-10-16]
11	16	Developers including PayPal and Epic Games are questioning whether a potential U.S. an...	[]	[Apple, PayPal, Epic Games]	[2026-10-16]
12	17	iPhone Duo delivery estimates slipped into late January 2027 within an hour of preorde...	[]	[Apple]	[2026-10-16, 2027, late January]

4.5 Summarize: the digest

[9]:
```
import ast


def digest_bullets(summary):
    """The digest summary as Markdown. This run's model grouped the
    bullets as {"theme", "points"} dicts, which the chain stored as
    text; show them as a theme with its points."""
    lines = []
    for line in str(summary).splitlines():
        try:
            item = ast.literal_eval(line.lstrip("- "))
        except (ValueError, SyntaxError):
            lines.append(line)
            continue
        if isinstance(item, dict) and "points" in item:
            lines.append(f"- **{item.get('theme', '')}**")
            lines += [f"    - {pt}" for pt in item["points"]]
        else:
            lines.append(line)
    return "\n".join(lines)


d = chain["digest"]
display(md(
    f"**{d['headline']}**\n\n{digest_bullets(d['summary'])}\n\n"
    f"**Catalysts:** {'; '.join(map(str, d['catalysts']))}\n\n"
    f"**Risks:** {'; '.join(map(str, d['risks']))}"
))
print("Sentiment:", d["sentiment_counts"])
print("Topics:", d["topic_counts"])
print("Unknown citations:", d["unknown_citations"] or "none")
```

Apple reaches $5 trillion valuation milestone with strong demand for new foldable iPhone,

though product pricing and legal challenges present headwinds.

- **Product Momentum**
  - Apple's $1,999 Foldable iPhone sold out in minutes with deliveries slipping to January [9], and iPhone Duo preorders delayed into late January 2027 [17], signaling strong consumer demand [11].
  - Apple launched HomeView smart home device using Siri AI technology [5], though the device faces app availability concerns [14].
  - New Apple TV 4K and HomePod mini price increases drew criticism for being more striking than new features [12].
- **Valuation & Leadership**
  - Apple's market value grew from $350 billion to $5 trillion under Tim Cook's 15-year tenure [4], with new CEO John Ternus now leading the company's next chapter [4].
  - Apple hit record closing prices ahead of November 2 earnings report [10], and maintains competitive positioning versus other mega-cap companies [1].
  - Analysts project positive returns from current Apple valuations through 2031 [7].
- **Risks**
  - Apple's HomeView and Siri AI initiatives are linked to a $250 million settlement with iPhone users, raising questions about technology reliability [5].
  - Potential U.S. antitrust settlement under scrutiny from developers including Epic Games and PayPal regarding iPhone control restrictions [16].

Catalysts: Foldable iPhone demand exceeding supply expectations [9][17]; Siri AI monetization through premium features [10]; Strong leadership transition to John Ternus [4]; November 2 earnings report potential upside [10]

Risks: Siri AI technology reputational concerns from prior settlement [5]; Potential antitrust restrictions on iPhone ecosystem [16]; Product pricing power erosion amid feature criticism [12]; HomeView app availability limiting smart home ecosystem [14]

```
Sentiment: {'positive': 6, 'neutral': 3, 'negative': 4}
Topics: {'product': 7, 'analyst_rating': 3, 'other': 2, 'legal': 1}
Unknown citations: none
```

The digest's three themes cite articles by id, and every citation matched a kept article ("Unknown citations: none"). Sentiment across the 13 relevant articles is mixed, 6 positive, 3 neutral and 4 negative, which matches the headline: strong demand for the foldable iPhone against pricing and legal headwinds. The agent's `get_news_digest` tool runs this same chain. In Section 7 the model asked it for 30 days of news instead of the 7 used here, so the agent's digest is built the same way from a wider window.

4.6 Cost and timing per step

[10]:
```python
# This run's numbers are zero on an offline replay; the live run's
# timings and token usage are read back from data/runs/.
live = live_runs("prompt_chain")[-1]
print("Live run:", live["run_at"])
print("Seconds per step:", live["timings"])
print("Token usage:", live["usage"])
```

```
Live run: 2026-10-18T21:45:36+00:00
Seconds per step: {'ingest': 0.18, 'preprocess': 0.03, 'classify': 8.61, 'extract': 11.79, 'summarize':
6.36}
Token usage: {'calls': 4, 'input_tokens': 6817, 'output_tokens': 3098}
```

The chain made 4 LLM calls on its live run, with 6,817 input and 3,098 output tokens, and took about 27

seconds. Classify and extract take most of that time (8.61 and 11.79 seconds), while ingest and preprocess together take about a fifth of a second.

5. Workflow 2: Routing

Routing sends each piece of content about the ticker to one of three specialists (`src/finagent/workflows/routing.py`):

Specialist	Handles
earnings	EPS vs. estimates, financial statements, earnings news
news	company events: products, legal and regulatory, management
market	price action, valuation and analyst targets, macro

Items are the prompt chain's relevant articles plus one item per structured tool (price summary, company info, financials, earnings, macro). The router works in two stages:

1. **Rules** route structured data by its kind and route articles by the topic label the prompt chain's classify step already assigned. This is free and deterministic, and the reason is easy to state.
2. **LLM router**: only articles labeled `other` go to the model, in one batched call. It returns a specialist and a one-sentence reason for each. An answer naming an unknown specialist falls back to the news analyzer and is marked `default`.

Each specialist gets only its own items. The numbers it reasons over (beat rate, average surprise, revenue growth, margins, position in the 52-week range, upside to the analyst target) are computed in Python. The LLM interprets them and returns a signal, a confidence, and key points. A tool that fails (missing key, rate limit) is skipped and listed in `errors`, so routing still runs on whatever is available.

5.1 Route the content

[11]:
```
routed = routing.run_routing(TICKER, chain=chain, save=LIVE)
decisions = pd.DataFrame(routed["decisions"])
print("Skipped tools:", routed["errors"] or "none")
decisions[["item_id", "kind", "specialist", "method", "reason"]]
```

```
Skipped tools: none
```

[11]:

	item_id	kind	specialist	method	reason
0	news-1	news	market	rule	classified as analyst_rating by the prompt chain
1	news-2	news	market	rule	classified as analyst_rating by the prompt chain
2	news-4	news	market	llm	Discusses Apple's market capitalization growth and valuation trajectory over time.
3	news-5	news	news	rule	classified as product by the prompt chain
4	news-6	news	news	llm	Covers management changes and business partnerships involving Apple's new CEO.
5	news-7	news	market	rule	classified as analyst_rating by the prompt chain
6	news-9	news	news	rule	classified as product by the prompt chain
7	news-10	news	news	rule	classified as product by the prompt chain
8	news-11	news	news	rule	classified as product by the prompt chain
9	news-12	news	news	rule	classified as product by the prompt chain
10	news-14	news	news	rule	classified as product by the prompt chain
11	news-16	news	news	rule	classified as legal by the prompt chain

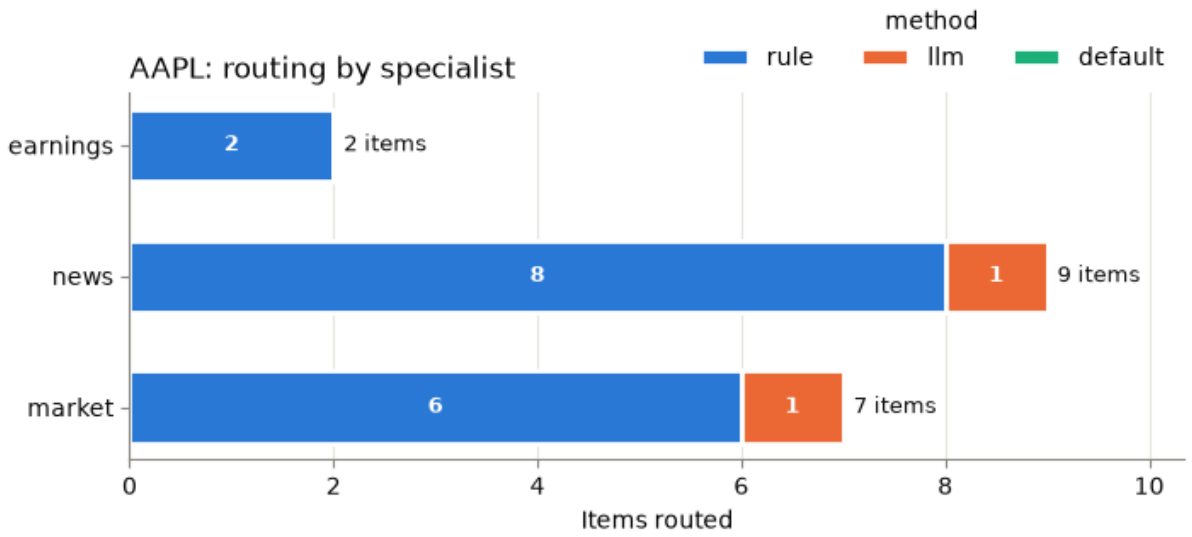
	item_id	kind	specialist	method	reason
12	news-17	news	news	rule	classified as product by the prompt chain
13	price_summary	price_summary	market	rule	structured price_summary data
14	company_info	company_info	market	rule	structured company_info data
15	financials	financials	earnings	rule	structured financials data
16	earnings	earnings	earnings	rule	structured earnings data
17	macro	macro	market	rule	structured macro data

Rules routed 16 of the 18 items. The two articles the chain had labeled `other` went to the LLM router: article 4, on Apple's market value growing to $5 trillion under Tim Cook, went to the market specialist ("Discusses Apple's market capitalization growth and valuation trajectory over time"), and article 6, on the new CEO's partnership with a former rival, went to the news specialist. The five structured tool results are routed by their kind, and no tool failed, so the list of skipped tools is empty.

5.2 Routing distribution

[12]:
```
# Items per specialist, split by how they were routed. Method gets the
# categorical colors; specialists are on the axis, so color is not the
# only cue.
methods = ["rule", "llm", "default"]
counts = (decisions.groupby(["specialist", "method"]).size()
          .unstack(fill_value=0)
          .reindex(index=list(routing.SPECIALISTS), columns=methods,
                   fill_value=0))

fig, ax = plt.subplots(figsize=(8, 2.8))
left = np.zeros(len(counts))
for color, m in zip(SERIES, methods):
    vals = counts[m].to_numpy()
    ax.barh(counts.index, vals, left=left, color=color, height=0.55,
            edgecolor="white", linewidth=2, label=m)
    for y, (l, v) in enumerate(zip(left, vals)):
        if v:
            ax.text(l + v / 2, y, str(v), ha="center", va="center",
                    color="white", fontsize=9, fontweight="bold")
    left += vals
for y, total in enumerate(left):
    ax.text(total + 0.1, y, f"{int(total)} items", va="center",
            color=INK, fontsize=9)
ax.invert_yaxis()
ax.set(xlabel="Items routed", title=f"{TICKER}: routing by specialist")
ax.set_xlim(0, left.max() * 1.15)
ax.legend(title="method", frameon=False, ncols=3, loc="lower right",
          bbox_to_anchor=(1, 1))
style(ax, grid_axis="x")
plt.show()
```



5.3 Specialist views

[13]:
```
pd.DataFrame([
    {"specialist": name, "items": a["n_items"], "signal": a["signal"],
     "confidence": a["confidence"], "summary": a["summary"]}
    for name, a in routed["analyses"].items()
]).style.set_properties(subset=["summary"], **{"text-align": "left"})
```

[13]:

	specialist	items	signal	confidence	summary
0	earnings	2	bullish	high	Apple demonstrates exceptional execution with a perfect 8-for-8 EPS beat streak, including a 7.45% surprise on the latest quarter. Revenue growth of 6.43% combined with a robust 26.92% net margin and strong free cash flow of $98.8B indicate healthy operational performance and financial strength.
1	news	9	neutral	medium	Apple shows mixed near-term signals: strong demand for premium products (foldable iPhone sold out, deliveries delayed to January) offsets concerns about pricing power, product execution risks, and ongoing legal/regulatory headwinds. The $250M Siri settlement reputational risk combined with HomeView app limitations and antitrust scrutiny present near-term overhang despite record stock price.
2	market	7	neutral	medium	Apple demonstrates strong price momentum (36.6% YTD return, hitting 52-week highs) with positive analyst sentiment, but valuation is stretched at 39.4x trailing PE with -4.1% downside to analyst targets. Rising Treasury yields (up 105 bps YoY) and elevated forward PE of 35.9x present headwinds despite improving macro conditions.

[14]:
```
for name, a in routed["analyses"].items():
    points = "\n".join(f"- {p}" for p in a["key_points"])
    display(md(f"**{name}** ({a['signal']}):\n\n{points}"))
live = live_runs("routing")[-1]
print("Live run seconds per stage:", live["timings"])
print("Live run token usage:", live["usage"])
```

earnings (bullish):

- Perfect 8-quarter EPS beat streak with 4.78% average surprise

- Latest quarter EPS beat: $2.02 reported vs $1.88 estimated (+7.45%)
- Net margin of 26.92% reflects pricing power and operational efficiency
- Free cash flow of $98.8B supports capital returns and strategic investments
- Revenue growth of 6.43% with YoY EPS growth from $1.57 to $2.02 (+28.7%)

news (neutral):

- $1,999 Foldable iPhone sold out in minutes with January delivery delays [news-9], indicating strong demand
- $250M settlement tied to Siri AI technology now deployed in new HomeView device raises reputational risk [news-5]
- Price increases on Apple TV 4K and HomePod mini 'much more striking' than new features [news-12]
- HomeView faces app availability problems [news-14]
- Antitrust settlement scope questioned by developers including Epic Games [news-16]

market (neutral):

- Stock at 52-week high with 1-year return of 36.64% showing strong momentum [price data]
- Trailing PE of 39.39x and forward PE of 35.86x reflect premium valuation relative to peers [valuation data]
- Analyst consensus 'buy' rating but mean price target of $329.38 implies -4.1% downside to current $343.46 [valuation data]
- 10-year Treasury yield up 105 bps YoY to 5.1%, pressuring high-multiple tech valuations [macro data]
- Analyst sentiment positive with comparisons highlighting record highs, though no specific price targets or growth forecasts provided [news-2, news-4]

```
Live run seconds per stage: {'collect': 2.35, 'route': 1.35, 'analyze': 13.0}
Live run token usage: {'calls': 4, 'input_tokens': 4281, 'output_tokens': 1151}
```

The specialists disagree, which is what routing is for. The earnings specialist is bullish with high confidence on a perfect 8-quarter beat record and a 7.45% surprise in the latest quarter. The market specialist is neutral: the stock sits at its 52-week high after a 36.64% one-year return, but at 39.39 times trailing earnings the analysts' mean target of $329.38 is 4.1% below the price. The news specialist is also neutral, weighing the sold-out foldable iPhone against the HomeView app problem, the price increases and the antitrust question. Each specialist saw only its own items, so each view rests on different evidence. The market specialist's summary calls the 36.6% figure a "YTD return", but it is the one-year return in its facts; the same label reappears in the brief (6.3).

6. Workflow 3: Evaluator-Optimizer

The evaluator-optimizer turns the three specialist views from routing into one investment brief and improves it in a loop (`src/finagent/workflows/evaluator_optimizer.py`):

1. **Generate**: the generator drafts a brief (view, thesis, evidence, risks, what would change the view) from the specialists' facts and signals.
2. **Evaluate**: a separate, strict "research director" prompt scores the draft from 1 to 5 on five criteria: coverage, evidence, balance, actionability, and clarity. It also writes specific fixes.
3. **Ground**: a sixth criterion, *grounding*, is computed in Python rather than by the LLM. Every number in the draft is matched against the source facts at the precision it was written with, after rescaling (fraction to %, raw to K/M/B/T). Numbers that can't be matched are put at the top of the feedback. The "What would change our view" section is skipped, since its numbers are hypothetical triggers rather than claims. LLM graders are poor at catching invented figures, and a regex is not.
4. **Refine**: the optimizer revises the draft using the feedback. The loop stops when a draft passes or after `max_iters` drafts. To pass, the overall score (mean of the six criteria / 5) must reach the

threshold **and** every criterion must score at least `min_score` (default 4), so a strong area can't hide a weak one.

A revision can score lower than the draft it revised, so the loop returns the best-scoring draft, not the last one. All three roles go through the disk cache, so an offline rerun replays identical drafts and scores.

6.1 Run the loop

Two details of the evaluator changed after our first live run (Section 9 explains why). The grader's prompt now defines each score level, for example 3 means something is missing, mislabeled or not backed by the facts. From the second iteration on, the grader also sees its own previous feedback, checks whether each point was addressed, and raises new points only for errors or missing required elements. The optimizer is told not to invent data that a feedback point asks for.

[15]:
```
context = eo.context_from_routing(routed)
loop = eo.run_loop(context, max_iters=3, threshold=THRESHOLD, save=LIVE)
print(
    f"{len(loop['history'])} drafts; best = iteration "
    f"{loop['best_iteration']}, passed: {loop['passed']}"
)
```

```
2 drafts; best = iteration 2, passed: True
```

6.2 Scores per iteration

[16]:
```
scores = pd.DataFrame(
    [{"iteration": h["iteration"], **h["scores"],
      "overall": h["overall"], "words": h["words"],
      "ungrounded": len(h["grounding"]["ungrounded"])}
     for h in loop["history"]]
).set_index("iteration")
scores
```

[16]:

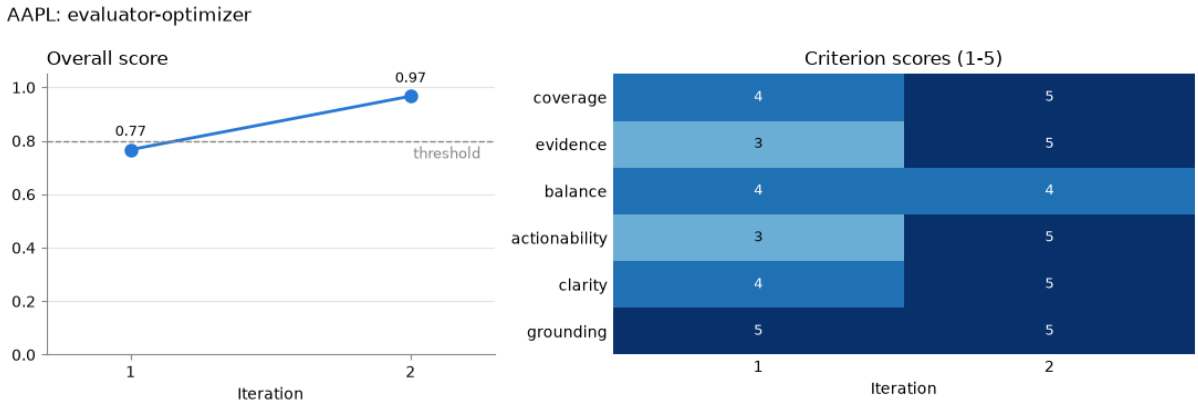
iteration	coverage	evidence	balance	actionability	clarity	grounding	overall	words	ungrounded
1	4.0	3.0	4.0	3.0	4.0	5.0	0.767	377	0
2	5.0	5.0	4.0	5.0	5.0	5.0	0.967	443	0

[17]:
```
# Left: overall score against the pass threshold. Right: each
# criterion per iteration on one sequential scale (darker = higher), so
# the reader sees which criteria the feedback actually moved.
hist = loop["history"]
iters = [h["iteration"] for h in hist]
criteria = list(hist[0]["scores"])
grid = np.array([[h["scores"][c] for h in hist] for c in criteria])

fig, (ax1, ax2) = plt.subplots(
    1, 2, figsize=(11, 3.8), gridspec_kw={"width_ratios": [1, 1.3]}
)
ax1.plot(iters, [h["overall"] for h in hist], color=SERIES[0], lw=2,
         marker="o", ms=8)
ax1.axhline(loop["threshold"], color=MUTED, ls="--", lw=1)
ax1.text(iters[-1] + 0.25, loop["threshold"] - 0.02, "threshold",
         ha="right", va="top", color=MUTED, fontsize=9)
for i, h in zip(iters, hist):
    ax1.annotate(f"{h['overall']:.2f}", (i, h["overall"]),
                 textcoords="offset points", xytext=(0, 9),
                 ha="center", fontsize=9, color=INK)
```

```
ax1.set(xticks=iters, xlim=(iters[0] - 0.3, iters[-1] + 0.3),
        ylim=(0, 1.05), xlabel="Iteration",
        title="Overall score")
style(ax1)

ax2.imshow(grid, cmap="Blues", vmin=1, vmax=5, aspect="auto")
ax2.set(xticks=range(len(iters)), xticklabels=iters,
        yticks=range(len(criteria)), yticklabels=criteria,
        xlabel="Iteration", title="Criterion scores (1-5)")
for (r, c), v in np.ndenumerate(grid):
    ax2.text(c, r, f"{v:g}", ha="center", va="center", fontsize=9,
             color="white" if v >= 3.5 else INK)
ax2.tick_params(length=0)
for s in ax2.spines.values():
    s.set_visible(False)
fig.suptitle(f"{TICKER}: evaluator-optimizer", x=0.01, ha="left")
fig.tight_layout()
plt.show()
```



The first draft failed at 0.767. Its average was below the 0.8 threshold, and two criteria, evidence and actionability, scored 3, below the minimum of 4. The second draft passed at 0.967 with five of the six criteria at 5. Grounding stayed at 5 in both iterations because every number in both drafts was found in the specialists' facts. The revision is also longer, 443 words against 377, and both exceed the 350-word limit in the brief's format (the count includes Markdown markers such as bullets). The grader did not penalize the length.

6.3 What the evaluator asked for, and the final brief

[18]:
```
first = loop["history"][0]
print("Iteration 1 feedback:")
for f in first["feedback"]:
    print(wrap(f, indent=" - "))
print("Ungrounded numbers in draft 1:",
      first["grounding"]["ungrounded"] or "none")
display(md(loop["final"]))
```

```
Iteration 1 feedback:
 - CRITICAL - Evidence gap on leadership transition risk: Brief claims 'Leadership transition from
   Tim Cook to John Ternus creates execution uncertainty' but cites only news-6 (neutral sentiment)
   without explaining why this is material to investment thesis. Either provide specific execution
   metrics this could impact or remove/downgrade as a primary risk.
```

```
 - MISSING - Current stock price context: Brief states analyst targets imply '-4.1% downside' but
   never explicitly states current price ($343.46) until buried in valuation section. Lead with: 'At
   $343.46, the stock trades at 39.4x trailing PE with consensus target of $329.38, implying…' for
   immediate clarity.
 - EVIDENCE - Macro backdrop incomplete: Brief cites 10-year Treasury up 105 bps to 5.1% but omits
   that federal funds rate actually fell 47 bps YoY (3.75% vs 4.22%), creating mixed macro signal.
   Acknowledge this contradiction-easier monetary policy partially offsets yield headwind for growth
   stocks.
 - ACTIONABLE - Sell trigger needs market condition: Current Sell trigger ('Antitrust settlement
   materially restricts App Store control') is vague. Quantify: specify what % reduction in App
   Store revenue would trigger Sell, or define 'material' with reference to current 26.92% net
   margin.
 - BALANCE - Underweight bull case on pricing power: Brief cites foldable iPhone sellout as demand
   validation but doesn't address whether $1,999 premium pricing is sustainable or a one-time halo
   effect. News-12 raises pricing fatigue risk; Add bull counterpoint: premium positioning
   historically holds at Apple or supply constraint explains sellout.
Ungrounded numbers in draft 1: none
```

AAPL investment brief

View: Hold, with medium conviction

Thesis

At $343.46, Apple trades at 39.4x trailing PE with consensus target of $329.38, implying 4.1% downside. While exceptional operational execution—perfect 8-quarter EPS beat streak and 26.92% net margin—validates pricing power, stretched valuation combined with near-term product execution risks and pricing fatigue concerns offset upside potential. Foldable iPhone demand ($1,999 sold out in minutes [news-9]) is offset by questions about premium pricing sustainability as feature improvements lag price increases [news-12].

Evidence

Earnings & Fundamentals (Bullish) - Perfect 8-quarter EPS beat streak with 7.45% surprise on latest quarter ($2.02 vs. $1.88 estimated) - Net margin of 26.92% and free cash flow of $98.8B demonstrate financial strength - Revenue growth of 6.43% with YoY EPS growth of 28.7% ($1.57 to $2.02)

Valuation (Cautionary) - Trailing PE of 39.4x and forward PE of 35.9x represent premium valuations at 52-week highs with 36.6% YTD return - Analyst consensus "buy" with mean target of $329.38 implies limited downside to current price

News & Execution (Mixed) - Foldable iPhone at $1,999 sold out in minutes with January delivery delays [news-9], signaling strong near-term demand - Price increases on Apple TV 4K and HomePod mini "much more striking" than new features [news-12]; premium positioning has historically held at Apple, but pricing fatigue risk warrants monitoring - HomeView faces app availability problems [news-14], creating execution risk for new ecosystem - Reputational concern: Apple deploying Siri AI technology in HomeView that was involved in $250M settlement [news-5] - Antitrust settlement scope questioned by developers including Epic Games [news-16], creating potential regulatory overhang

Macro (Mixed Signal) - 10-year Treasury yield up 105 bps YoY to 5.1%, pressuring high-multiple tech valuations - However, federal funds rate fell 47 bps YoY (3.75% vs. 4.22%), indicating easier monetary policy that partially offsets yield headwind for growth stocks - Unemployment improved to 4.2% and VIX down to 16.46, supporting risk appetite

Risks

- Valuation compression if growth disappoints relative to 39.4x multiple or 10-year yields rise further
- Regulatory constraints: if antitrust settlement reduces App Store revenue >10% from current 26.92% net margin, material earnings impact
- Consumer pricing fatigue if feature improvements continue lagging price increases
- HomeView execution risk and insufficient app ecosystem adoption
- 24.5% annualized volatility increases downside risk in market corrections

What would change our view

- **Upgrade to Buy:** Analyst price targets rise above $360 with FY2027 EPS guidance ≥30% YoY growth, or HomeView achieves 5M+ active users within 12 months
- **Downgrade to Sell:** Antitrust settlement reduces App Store revenue contribution by ≥15% from current levels, or Q1 FY2027 guidance misses foldable iPhone demand expectations by ≥20%

The feedback maps directly onto the changes in the passing draft. The grader asked the brief to lead with the current price, and the thesis now opens "At $343.46, Apple trades at 39.4x trailing PE". It pointed out that the federal funds rate fell 47 basis points while the 10-year yield rose, and the macro section is now headed "Mixed Signal" and says so. It called the sell trigger vague, and both triggers now carry levels (App Store revenue down at least 15%, guidance missing by at least 20%). Those levels are hypothetical thresholds rather than facts, which is why the grounding check skips that section.

Two problems survived both graders. The brief calls the 36.6% one-year return a "YTD return". It also cites revenue growth of 6.43% without naming the period: that figure is fiscal 2025 over fiscal 2024 from the annual statements, while the agent in Section 7 reports 16.4%, Yahoo's year-over-year figure for the latest quarter. Both numbers are correct for their period; only the agent's report says which period it means.

[19]:
```
# LLM calls this loop makes: one draft, one evaluation per iteration,
# one refinement between iterations. (The loop's prompts were tuned
# over several live runs, so no single saved run shows its full cost.)
n = len(loop["history"])
print(f"LLM calls: {1 + n + (n - 1)} (1 draft, {n} evaluations, "
      f"{n - 1} refinement)")
```

```
LLM calls: 4 (1 draft, 2 evaluations, 1 refinement)
```

7. Agent Functions

These are the four agent functions that make the research agent behave like an analyst instead of a one-shot LLM prompt.

[20]:
```
# The demo keeps its notes in memory/notebook_demo/, cleared first, so
# run 1 always starts with no notes and an offline replay matches the
# live run. Only this folder's JSON files are removed.
DEMO_MEMORY = config.MEMORY_DIR / "notebook_demo"
DEMO_MEMORY.mkdir(parents=True, exist_ok=True)
for old in DEMO_MEMORY.glob("*.json"):
    old.unlink()

run1 = research_agent.run(TICKER, memory_dir=DEMO_MEMORY, save=LIVE)
print(f"Run 1: {run1['plan']['source']} plan, {len(run1['trace'])} tool "
      f"calls over {run1['turns']} turns (stopped: {run1['stopped']}), "
      f"{len(run1['drafts'])} draft(s), {run1['recommendation']}")
```

```
Run 1: llm plan, 11 tool calls over 4 turns (stopped: model_finished), 3 draft(s), Hold (medium
conviction)
```

7.1 Planning

The planner creates a short task list before the agent starts calling tools. The list covers price context, valuation, earnings, news, macro, and risk review. If the model returns malformed JSON, the code falls back to a safe default plan so the run still remains deterministic.

[21]:
```
def plan_table(plan):
    tools = plan["step_tools"] + [[]] * len(plan["steps"])
    return pd.DataFrame(
        {"step": plan["steps"],
         "tools": [", ".join(t) for t in tools[:len(plan["steps"])]]},
        index=range(1, len(plan["steps"]) + 1),
    ).style.set_properties(**{"text-align": "left"})  # full step text


p1 = run1["plan"]
print("Plan source:", p1["source"], f"({p1['error']})" if p1["error"]
      else "")
plan_table(p1)
```

```
Plan source: llm
```

[21]:

	step	tools
1	Establish current price momentum and technical backdrop	get_price_summary, get_price_history
2	Assess company fundamentals, valuation multiples, and historical context	get_company_info, get_financials
3	Evaluate recent earnings performance vs. expectations and trend	get_earnings, get_recent_filings
4	Analyze recent news flow and investor sentiment	get_news_digest, get_news
5	Understand the macroeconomic environment and its relevance to AAPL	get_macro_snapshot, get_series
6	Review latest 10-Q or 10-K filing for forward guidance, risks, and capital allocation	get_filing_text
7	Synthesize findings into catalysts, risks, and Buy/Hold/Sell recommendation	

The plan's source is "llm": the planner's call succeeded and returned usable JSON, so the default plan was not needed. The model wrote seven steps that follow the areas the prompt asks for (price, fundamentals, earnings, news, macro, and the risks behind a rating) and added one of its own, reading the latest 10-Q or 10-K for guidance and risks. Each step names the tools that should answer it. These names reach the tool loop as suggestions only, and the model is free to call other tools. The last step, synthesis, needs no tool.

7.2 Dynamic Tool Use

The agent loop asks the model for tool calls, executes each tool through the registry, and adds the result back to the conversation. This allows later steps to depend on what was already learned, instead of making isolated single-shot calls. Tool failures are returned as structured error objects so the run can continue with the remaining facts.

[22]:
```
def trace_table(run):
    t = pd.DataFrame(run["trace"])
    t["args"] = t["args"].map(
        lambda a: ", ".join(f"{k}={v}" for k, v in a.items()))
    t["error"] = t["error"].fillna("").str.slice(0, 80)
    return t[["turn", "tool", "args", "ok", "chars", "truncated",
```

```
            "error"]]


trace = trace_table(run1)
print(f"{len(trace)} calls, {trace['tool'].nunique()} distinct tools, "
      f"{(~trace['ok']).sum()} errors")
trace
```

```
11 calls, 9 distinct tools, 0 errors
```

[22]:

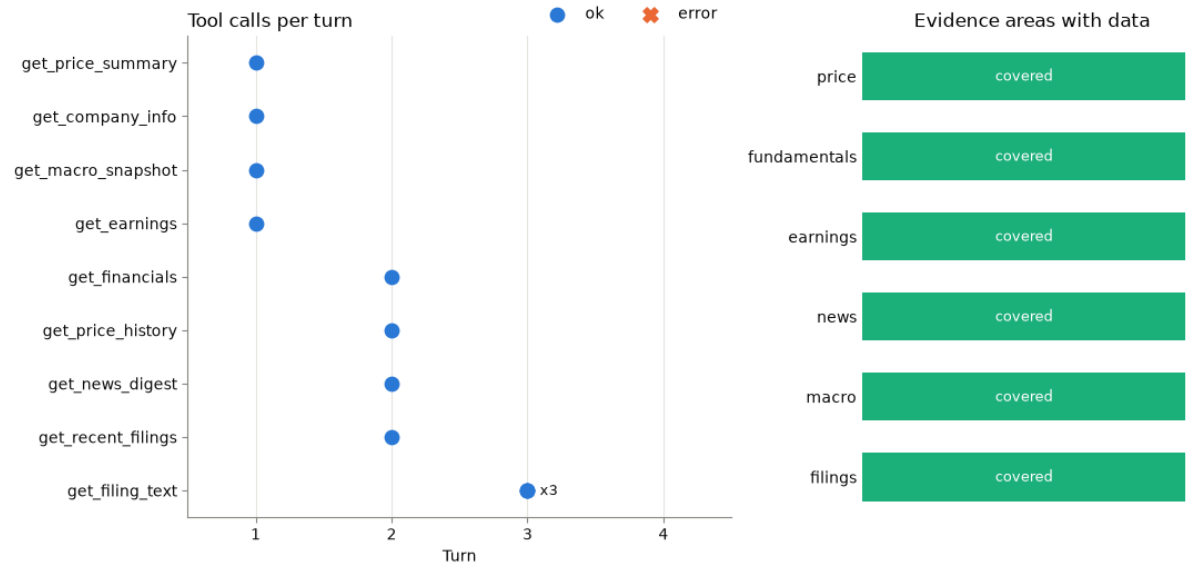
	turn	tool	args	ok	chars	truncated	error
0	1	get_price_summary	ticker=AAPL	True	221	False	
1	1	get_company_info	ticker=AAPL	True	1882	False	
2	1	get_macro_snapshot		True	687	False	
3	1	get_earnings	ticker=AAPL, quarters=8	True	1246	False	
4	2	get_financials	ticker=AAPL, quarterly=True, periods=5	True	2630	False	
5	2	get_price_history	ticker=AAPL, period=1y, interval=1mo	True	1378	False	
6	2	get_news_digest	ticker=AAPL, days=30, limit=30	True	4472	False	
7	2	get_recent_filings	ticker=AAPL, form=10-Q,10-K,8-K, limit=5	True	1144	False	
8	3	get_filing_text	url=https://www.sec.gov/Archives/edgar/data/320193/000032019326000020/aapl-20260627.ht...	True	6017	True	
9	3	get_filing_text	url=https://www.sec.gov/Archives/edgar/data/320193/000032019326000020/aapl-20260627.ht...	True	5290	False	
10	3	get_filing_text	url=https://www.sec.gov/Archives/edgar/data/320193/000032019326000020/aapl-20260627.ht...	True	6017	True	

[23]:

```
# Left: every tool call as a point, turn on x, tool on y (filled = ok,
# cross = error). Right: which evidence areas ended up with data.
tools = list(dict.fromkeys(trace["tool"]))
row = {t: i for i, t in enumerate(tools)}
fig, (ax1, ax2) = plt.subplots(
    1, 2, figsize=(11, 0.42 * len(tools) + 1.6),
    gridspec_kw={"width_ratios": [1.6, 1]})
ok, bad = trace[trace["ok"]], trace[~trace["ok"]]
ax1.scatter(ok["turn"], ok["tool"].map(row), s=80, color=SERIES[0],
            label="ok", zorder=3)
ax1.scatter(bad["turn"], bad["tool"].map(row), s=90, marker="X",
            color=SERIES[1], label="error", zorder=3)
ax1.set(yticks=range(len(tools)), yticklabels=tools,
        xticks=range(1, run1["turns"] + 1), xlabel="Turn",
        xlim=(0.5, run1["turns"] + 0.5), ylim=(len(tools) - 0.5, -0.5),
        title="Tool calls per turn")
# Several calls to one tool in one turn (different filing sections)
# would draw as one point, so label the count.
for (turn, tool), k in trace.groupby(["turn", "tool"]).size().items():
    if k > 1:
        ax1.annotate(f"x{k}", (turn, row[tool]), xytext=(8, -3),
                     textcoords="offset points", fontsize=9, color=INK)
ax1.legend(frameon=False, ncols=2, loc="lower right",
           bbox_to_anchor=(1, 1))
style(ax1, grid_axis="x")

areas = list(run1["coverage"])
covered = [run1["coverage"][a] for a in areas]
ax2.barh(areas, [1] * len(areas), height=0.6,
         color=[SERIES[2] if c else "#e6e5e0" for c in covered])
for y, c in enumerate(covered):
    ax2.text(0.5, y, "covered" if c else "missing", ha="center",
```

```
            va="center", fontsize=9, color="white" if c else INK)
ax2.set(xticks=[], ylim=(len(areas) - 0.5, -0.5),
        title="Evidence areas with data")
for s in ax2.spines.values():
    s.set_visible(False)
ax2.tick_params(length=0, labelcolor=INK)
fig.tight_layout()
plt.show()
```



The agent made 11 tool calls with 9 different tools over three turns and stopped in the fourth by replying without a tool call (`stopped: model_finished`). It batched independent calls: four in turn 1 and four in turn 2. Turn 3 shows why a loop is more useful than a fixed pipeline. The model read three sections of Apple's 10-Q (Risk Factors, Liquidity, and Management's Discussion) using the document URL that `get_recent_filings` had returned in turn 2, so the third turn depended on what the second one found. Two of those excerpts reached the 6,000-character limit per tool result and were truncated, which keeps the conversation within budget at the cost of some filing text. All six evidence areas have data. The agent did not call `get_news` or `get_series`, which overlap with the news digest and the macro snapshot it already had.

Recovering from a tool failure

[24]:
```
# A tool failure on purpose: for this one tool loop get_earnings
# returns an error (as an Alpha Vantage rate limit would), and the agent
# has to find earnings evidence with another tool. Same plan as run 1.
real_call = research_agent.registry.call


def earnings_down(name, args=None):
    if name == "get_earnings":
        return {"error": "RuntimeError: Alpha Vantage rate limit reached "
                         "(simulated for this demo)"}
    return real_call(name, args)


research_agent.registry.call = earnings_down
```

```
try:
    failed = research_agent.gather(TICKER, run1["plan"])
finally:
    research_agent.registry.call = real_call

failed_trace = trace_table(failed)
print(f"{len(failed_trace)} calls over {failed['turns']} turns, "
      f"{(~failed_trace['ok']).sum()} error(s); earnings covered: "
      f"{research_agent.coverage(failed['facts'])['earnings']}")
display(failed_trace[["turn", "tool", "args", "ok", "error"]])

# The tool lesson this run would save for every later run.
cover = research_agent.coverage(failed["facts"])
note = research_agent.lessons(TICKER, {}, {"gaps": []},
                              {**failed, "coverage": cover})["general"]
print("Tool lesson:", note[0]["note"] if note else "none")
```

```
11 calls over 4 turns, 1 error(s); earnings covered: True
```

	turn	tool	args	ok	error
0	1	get_price_summary	ticker=AAPL	True	
1	1	get_company_info	ticker=AAPL	True	
2	1	get_macro_snapshot		True	
3	1	get_earnings	ticker=AAPL, quarters=8	False	RuntimeError: Alpha Vantage rate limit reached (simulated for this demo)
4	2	get_price_history	ticker=AAPL, period=1y	True	
5	2	get_financials	ticker=AAPL, quarterly=False, periods=4	True	
6	2	get_recent_filings	ticker=AAPL, form=10-K,10-Q,8-K, limit=5	True	
7	2	get_news_digest	ticker=AAPL, days=30, limit=30	True	
8	3	get_filing_text	url=https://www.sec.gov/Archives/edgar/data/320193/000032019326000020/aapl-20260627.ht...	True	
9	3	get_filing_text	url=https://www.sec.gov/Archives/edgar/data/320193/000032019326000020/aapl-20260627.ht...	True	
10	3	get_series	series_id=DGS10, start=2024-10-16	True	

```
Tool lesson: get_earnings returned an error (RuntimeError: Alpha Vantage rate limit reached (simulated
for this demo)); get_financials worked instead.
```

With `get_earnings` returning an error in turn 1, the agent did not call it again, as its instructions say. The rest of the loop went ahead: in turn 2 it fetched the annual financial statements, which carry reported EPS and net income, so the earnings area is still covered, though without the analyst estimate and surprise that only Alpha Vantage provides. In turn 3 it read the 10-Q and also fetched the 10-year Treasury yield history (`get_series`, DGS10), which run 1 never called: with a different result in turn 1, the model chose a slightly different set of tools afterwards. The run also produced the general tool lesson shown above. In a real run it would be saved to `general.json` and shown to the planner on every later run, for any ticker. We did not save it here, so run 2 below is not affected by the simulated failure.

The report

[25]:
```
def show_report(r):
    def bullets(items):
        return "\n".join(f"- {i}" for i in items)
```

```
    display(md(
        f"**Recommendation:** {r['recommendation']}\n\n"
        f"**Summary.** {r['summary']}\n\n"
        f"**Thesis.** {r['investment_thesis']}\n\n"
        f"**Key facts (computed in Python):**\n\n"
        f"{bullets(r['key_facts'])}\n\n"
        f"**Upside:**\n\n{bullets(r['upside'])}\n\n"
        f"**Risks:**\n\n{bullets(r['risks'])}\n\n"
        f"**Catalysts:**\n\n{bullets(r['catalysts'])}\n\n"
        f"**Outlook.** {r['outlook']}"))


show_report(run1)
```

Recommendation: Hold (medium conviction)

Summary. Apple trades at $343.46 (as of October 16, 2026) with strong momentum—up 36.6% over the past year—driven by iPhone Duo sellouts and 8 consecutive quarterly earnings beats. The company maintains robust fundamentals with 28.7% YoY earnings growth and 27.2% net margins, though a 39.4x trailing P/E reflects premium valuation amid rising discount rates.

Thesis. Apple's operational excellence is evident in 8 consecutive quarterly earnings beats (averaging 4.8% surprise, latest 7.4% on July 30, 2026) and 16.4% YoY revenue growth in Q3 FY2026 (ended June 30, 2026). The $1,999 iPhone Duo sold out in minutes with January 2027 deliveries, validating premium hardware demand. However, a 39.4x trailing P/E remains elevated, and the 10-year Treasury yield at 5.1% (up 105 basis points year-over-year as of October 15, 2026) raises discount rates for tech equities. Leadership transition to new CEO John Ternus and HomeView ecosystem risks—including a $250 million Siri AI settlement and app compatibility questions—create execution risk during a critical product cycle.

Key facts (computed in Python):

- last close: 343.46 (as of 2026-10-16)
- 1-year return: +36.6%
- 52-week range: 246.03 to 343.46
- market cap: $5.01T
- trailing P/E: 39.4
- revenue growth (yoy): +16.4%
- analyst mean target: 329.38 (39 analysts)
- Effective federal funds rate (%): 3.75%
- 10-year Treasury yield (%): 5.1%
- latest EPS: 2.02 vs 1.88 estimated (+7.45% surprise), quarter ending 2026-06-30

Upside:

- iPhone Duo sellout momentum and January 2027 delivery ramp could accelerate Q1 FY2027 revenue, building on 16.4% YoY growth reported in Q3 FY2026 (ended June 30, 2026)
- Premium Siri AI services monetization offers operating leverage given 27.2% net margins (Q3 FY2026) and $31.9 billion free cash flow in latest quarter
- JP Morgan's $360 price target vs. current $343.46 implies 4.8% upside, though consensus analyst mean target is $329.38 (39 analysts), implying 4.2% downside vs. current price

Risks:

- HomeView device faces app compatibility questions and ecosystem adoption risks, compounded by Siri AI reputational concerns tied to $250 million settlement affecting new product credibility

- Leadership transition to new CEO John Ternus introduces execution risk during a critical product cycle
- Valuation multiple compression risk: 39.4x trailing P/E vs. 35.9x forward P/E remains elevated; 10-year Treasury yield at 5.1% (up 105 basis points year-over-year as of October 15, 2026) raises discount rates for tech equities
- iPhone demand concerns persist in analyst commentary despite foldable enthusiasm, suggesting broader smartphone market saturation risks

Catalysts:

- November 2, 2026 earnings report and forward guidance could validate or challenge foldable iPhone demand sustainability
- January 2027 iPhone Duo delivery ramp and Q1 FY2027 results will demonstrate whether $1,999 price point achieves volume and gross margin targets
- Premium AI Siri services launch timing and monetization metrics will signal whether services can offset iPhone hardware cycle dependency

Outlook. Apple is well-positioned for near-term product momentum from foldable iPhone success, but current valuation multiples offer limited margin of safety amid leadership transitions, rising real rates (10-year yield at 5.1%), and HomeView execution risks; Hold rating reflects balanced risk-reward at $343.46.

The recommendation is Hold with medium conviction, and the model wrote it from the gathered facts. The ten key facts are computed in Python from the tool results, so they cannot be misquoted. The model's text cites those facts and adds context from the news digest and the 10-Q, such as the $1,999 iPhone Duo selling out, the $250 million Siri settlement and the November 2 earnings date. This is the draft the agent kept after self-reflection (7.3).

It still contains one error that our checks missed. The last upside point says the consensus target implies "4.2% downside", but $329.38 against $343.46 is 4.1% (the market specialist in Section 5 says 4.1%). Grounding accepted 4.2 because the unemployment rate in the macro facts is 4.2%. The check asks whether a number appears anywhere in the facts, not whether it belongs to the claim.

7.3 Self-Reflection

After creating a first-pass brief, the agent reflects on coverage, evidence, recency, and balance. We also run a heuristic check for placeholder language such as generic summaries, empty catalysts, and missing recommendations. That gives us a concrete list of gaps even when the model output is thin.

[26]:
```
def reflection_table(run):
    return pd.DataFrame([{
        "draft": h["draft"],
        "coverage": h["coverage_score"],
        "evidence": h["evidence_score"],
        "recency": h["recency_score"],
        "balance": h["balance_score"],
        "overall": h["score"],
        "grounded": (h["grounding"] or {}).get("ratio"),
        "ungrounded": ", ".join(
            (h["grounding"] or {}).get("ungrounded", [])) or "none",
        "fixable": len(h.get("fixable", [])),
        "needs data": len(h.get("needs_data", [])),
        "revise": h["revise"],
    } for h in run["reflection_history"]]).set_index("draft")


refl = reflection_table(run1)
```

```
print(f"Revise when overall < {research_agent.REVISE_BELOW}, a number "
      "is ungrounded, or the critic lists a gap the facts can fix; "
      f"best draft kept: {run1['best_draft']}")
refl
```

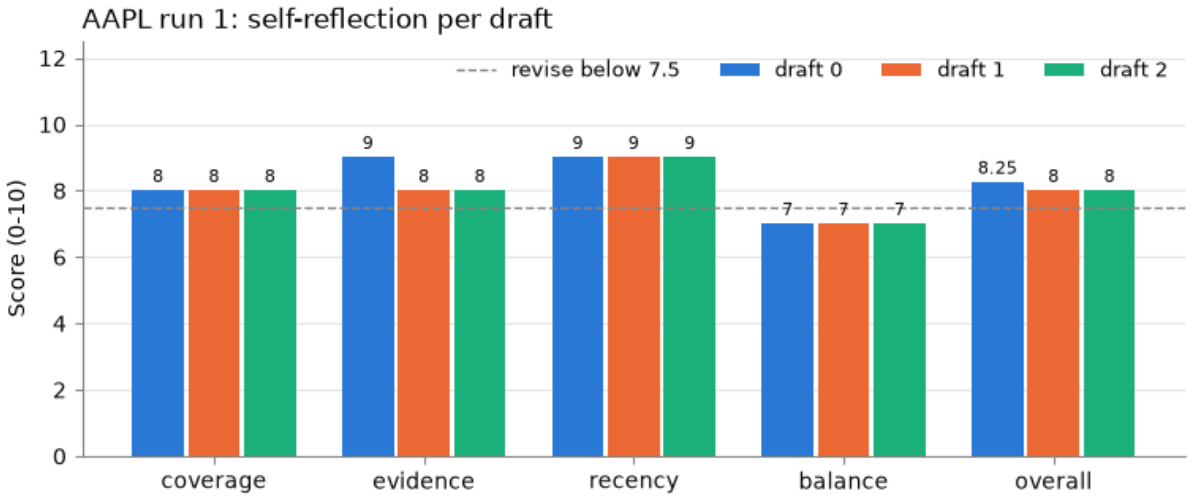
```
Revise when overall < 7.5, a number is ungrounded, or the critic lists a gap the facts can fix; best
draft kept: 2
```

[26]:

draft	coverage	evidence	recency	balance	overall	grounded	ungrounded	fixable	needs data	revise
0	8	9	9	7	8.25	1.000	none	3	7	True
1	8	8	9	7	8.00	0.973	3.5	3	5	True
2	8	8	9	7	8.00	1.000	none	0	4	False

[27]:

```
# Four criteria and the overall score for each draft, with the
# revision threshold. Drafts get the categorical colors in order.
cols = ["coverage", "evidence", "recency", "balance", "overall"]
x = np.arange(len(cols))
n = len(refl)
width = 0.8 / n
fig, ax = plt.subplots(figsize=(9, 3.4))
for i, (draft, rowv) in enumerate(refl.iterrows()):
    bars = ax.bar(x + (i - (n - 1) / 2) * width, rowv[cols].astype(float),
                  width * 0.92, color=SERIES[i % len(SERIES)],
                  label=f"draft {draft}")
    ax.bar_label(bars, fmt="%g", fontsize=8, padding=2, color=INK)
ax.axhline(research_agent.REVISE_BELOW, color=MUTED, ls="--", lw=1,
           label=f"revise below {research_agent.REVISE_BELOW}")
ax.set(xticks=x, xticklabels=cols, ylim=(0, 12.5), ylabel="Score (0-10)",
       title=f"{TICKER} run 1: self-reflection per draft")
ax.legend(frameon=False, ncols=n + 1, loc="upper right", fontsize=9)
style(ax)
plt.show()
```



Draft 0 scored 8.25, above the 7.5 revision threshold, and was revised anyway because the critic listed three gaps it could fix from facts the agent already had. Draft 1 scored 8.00 and introduced a number the facts do not contain: 3.5, the gap between the trailing and forward P/E (39.4 minus 35.9) that the critic

itself had suggested adding. The arithmetic is right, but grounding accepts only numbers that appear in the facts, so it flagged 3.5 and forced a second revision. Draft 2 scored 8.00 with every number grounded and nothing left to fix, so the loop stopped.

The agent kept draft 2 although draft 0 scored 0.25 higher, because a draft that needs no revision ranks above a higher-scoring draft that does. A quarter point is within the noise of a critic that scores each criterion in whole numbers, while the fixes are concrete (7.3 below shows them). Balance stayed at 7 in every draft: the critic's main complaint, that the upside rests on future events, cannot be fixed with the data the agent has.

Self-reflection and the evaluator-optimizer (Section 6) are both critique loops, but they differ in scope and in what happens next. Reflection is the agent judging its own report against the facts it gathered, on four criteria from 0 to 10, and its output also feeds memory: gaps that need new data become lessons for the next run. The evaluator-optimizer is a separate workflow with its own generator, evaluator and optimizer, a five-criterion rubric from 1 to 5 plus grounding, and a pass threshold, and it only improves the one brief.

[28]:
```
r0 = run1["reflection_history"][0]
print("Critic feedback on draft 0:")
print(wrap(r0["feedback"], indent="  "))
for key, label in (("fixable", "Fixable from the gathered facts"),
                   ("needs_data", "Needs data the run did not gather")):
    print(f"\n{label} ({len(r0[key])}):")
    for g in r0[key]:
        print(wrap(g, indent=" - "))
```

```
Critic feedback on draft 0:
  This is a well-structured report with strong factual grounding and recent data (as of October 16,
  2026). The analysis covers company fundamentals, market position, and macro factors appropriately.
  However, the balance between upside and downside could be stronger-the risks section addresses
  real concerns but some upside catalysts (foldable iPhone demand, AI services) rely on future
  validation rather than current execution proof.

Fixable from the gathered facts (3):
 - Forward P/E of 35.9x is cited as evidence that valuation multiples are compressed vs. trailing
   39.4x, but this comparison needs context: a 3.5-point gap suggests modest compression, not
   dramatic relief. The report could note whether this forward multiple remains elevated vs.
   historical Apple averages.
 - The report states '28.7% earnings growth' in the summary but provides 'earningsGrowth: 0.287'
   from company info without specifying the period (YoY? Forward?). The facts show Q3 FY2026 EPS of
   $2.02 vs. year-ago $1.57, which is +28.7% YoY, so the claim is correct but the period should be
   explicit in the summary.
 - AI services monetization is cited as upside but no specific revenue contribution or timeline is
   provided; the report should note this is speculative against the concrete foldable iPhone sellout
   evidence.

Needs data the run did not gather (7):
 - HomeView device sales, pre-orders, or launch date details to validate ecosystem adoption risk
   claims beyond 'app compatibility questions'.
 - JP Morgan's full price target rationale and date to contextualize the $360 target vs. consensus
   $329.38 and current $343.46 price.
 - Historical Apple trailing P/E multiples (3-year, 5-year average) to properly frame whether 39.4x
   is 'elevated even by historical standards' as claimed.
 - New CEO John Ternus's specific background, appointment date, and any public statements on product
   strategy to substantiate 'execution risk' beyond vague 'leadership transition' language.
 - iPhone Duo's $1,999 pricing vs. competitor foldables (Samsung Galaxy Z Fold, etc.) to assess
   whether demand reflects Apple's premium positioning or broader foldable adoption.
 - Specific timeline and pricing for 'premium AI Siri capabilities' monetization to test credibility
   of services upside thesis.
 - Details on the $250 million Siri settlement to assess whether HomeView reputational risk is
```

material or transient.

[29]:
```
# What the revision changed: each report field that differs between
# the first draft and the draft that was kept.
d0, kept = run1["drafts"][0], run1["drafts"][run1["best_draft"]]
fields = ["recommendation", "summary", "investment_thesis", "upside",
          "risks", "catalysts", "outlook"]
changed = [f for f in fields if d0[f] != kept[f]]
if len(run1["drafts"]) == 1:
    print("No revision: draft 0 already met the bar (score "
          f"{r0['score']}, no ungrounded numbers, nothing fixable).")
elif not changed:
    print("Revisions were made, but draft 0 scored best and was kept.")
for f in changed:
    print(f"== {f}")
    for label, d in (("first", d0), ("kept ", kept)):
        text = d[f] if isinstance(d[f], str) else " | ".join(d[f])
        print(wrap(text, indent=f"  {label}: "))
    print()
```

```
== summary
  first: Apple trades at $343.46 (as of October 16, 2026) with strong momentum-up 36.6% over the
         past year-driven by record valuations, foldable iPhone sellouts, and consistent earnings
         beats. The company maintains robust fundamentals with 28.7% earnings growth and 27.6% net
         margins, though valuation multiples (39.4x trailing P/E) reflect premium positioning and
         near-term execution risks.
  kept : Apple trades at $343.46 (as of October 16, 2026) with strong momentum-up 36.6% over the
         past year-driven by iPhone Duo sellouts and 8 consecutive quarterly earnings beats. The
         company maintains robust fundamentals with 28.7% YoY earnings growth and 27.2% net margins,
         though a 39.4x trailing P/E reflects premium valuation amid rising discount rates.

== investment_thesis
  first: Apple's 8-quarter perfect earnings beat streak (averaging 4.78% surprises, latest beat
         7.45% on June 30, 2026) and strong revenue growth of 16.4% year-over-year demonstrate
         operational excellence under current management. The foldable iPhone (iPhone Duo) sellout
         in minutes with January 2027 deliveries signals robust premium product demand and validates
         Apple's hardware innovation trajectory. However, a 39.4x trailing P/E ratio-elevated even
         by historical standards-coupled with leadership transition risk under new CEO John Ternus
         and HomeView ecosystem concerns leaves limited margin for error. The $5 trillion market
         capitalization and analyst consensus 'buy' rating with a $329.38 mean price target suggest
         the market has priced in significant upside, warranting a cautious stance at current
         levels.
  kept : Apple's operational excellence is evident in 8 consecutive quarterly earnings beats
         (averaging 4.8% surprise, latest 7.4% on July 30, 2026) and 16.4% YoY revenue growth in Q3
         FY2026 (ended June 30, 2026). The $1,999 iPhone Duo sold out in minutes with January 2027
         deliveries, validating premium hardware demand. However, a 39.4x trailing P/E remains
         elevated, and the 10-year Treasury yield at 5.1% (up 105 basis points year-over-year as of
         October 15, 2026) raises discount rates for tech equities. Leadership transition to new CEO
         John Ternus and HomeView ecosystem risks-including a $250 million Siri AI settlement and
         app compatibility questions-create execution risk during a critical product cycle.

== upside
  first: Foldable iPhone (iPhone Duo) sellout momentum and January 2027 delivery ramp could drive Q2
         FY2027 revenue acceleration, building on 16.4% YoY growth reported in Q3 FY2026 (June 30,
         2026) | AI services monetization via premium Siri capabilities offers operating leverage
         given 27.6% current net margins and strong free cash flow of $31.9 billion in latest
         quarter | JP Morgan's $360 price target (vs. current $343.46) implies 4.8% upside and
         represents analyst confidence despite iPhone demand concerns
  kept : iPhone Duo sellout momentum and January 2027 delivery ramp could accelerate Q1 FY2027
         revenue, building on 16.4% YoY growth reported in Q3 FY2026 (ended June 30, 2026) | Premium
         Siri AI services monetization offers operating leverage given 27.2% net margins (Q3 FY2026)
```

```
        and $31.9 billion free cash flow in latest quarter | JP Morgan's $360 price target vs.
        current $343.46 implies 4.8% upside, though consensus analyst mean target is $329.38 (39
        analysts), implying 4.2% downside vs. current price

== risks
  first: HomeView device faces app compatibility and ecosystem adoption risks, compounded by Siri AI
         reputational concerns tied to $250 million settlement affecting new product credibility |
         Leadership transition to new CEO John Ternus introduces execution risk during a critical
         product cycle, particularly given partnerships with Tim Cook's former rivals | Valuation
         multiple compression risk: 39.4x trailing P/E significantly exceeds the stock's forward P/E
         of 35.9x, and 10-year Treasury yield of 5.1% (up 105 basis points year-over-year as of
         October 15, 2026) raises discount rates for tech equities | iPhone demand concerns persist
         in analyst commentary despite recent foldable enthusiasm, suggesting broader smartphone
         market saturation risks
  kept : HomeView device faces app compatibility questions and ecosystem adoption risks, compounded
         by Siri AI reputational concerns tied to $250 million settlement affecting new product
         credibility | Leadership transition to new CEO John Ternus introduces execution risk during
         a critical product cycle | Valuation multiple compression risk: 39.4x trailing P/E vs.
         35.9x forward P/E remains elevated; 10-year Treasury yield at 5.1% (up 105 basis points
         year-over-year as of October 15, 2026) raises discount rates for tech equities | iPhone
         demand concerns persist in analyst commentary despite foldable enthusiasm, suggesting
         broader smartphone market saturation risks

== catalysts
  first: November 2, 2026 earnings report and forward guidance (imminent, per news digest) could
         validate or challenge foldable iPhone demand sustainability | January 2027 Foldable iPhone
         delivery ramp and Q1 FY2027 results will demonstrate whether $1,999 price point achieves
         volume and gross margin targets | Premium AI Siri services launch timing and monetization
         metrics will signal whether services can offset iPhone hardware cycle dependency
  kept : November 2, 2026 earnings report and forward guidance could validate or challenge foldable
         iPhone demand sustainability | January 2027 iPhone Duo delivery ramp and Q1 FY2027 results
         will demonstrate whether $1,999 price point achieves volume and gross margin targets |
         Premium AI Siri services launch timing and monetization metrics will signal whether
         services can offset iPhone hardware cycle dependency

== outlook
  first: Apple is well-positioned for near-term product momentum driven by foldable iPhone success
         and AI service expansion, but current valuation multiples offer limited margin of safety
         amid leadership transitions, rising real rates (10-year yield at 5.1%), and HomeView
         execution risks; Hold rating reflects balanced risk-reward at $343.46.
  kept : Apple is well-positioned for near-term product momentum from foldable iPhone success, but
         current valuation multiples offer limited margin of safety amid leadership transitions,
         rising real rates (10-year yield at 5.1%), and HomeView execution risks; Hold rating
         reflects balanced risk-reward at $343.46.
```

The critic split its gaps as asked. One of the three fixable gaps is a real precision problem: the summary said "28.7% earnings growth" without a period, and the critic found the matching facts (EPS of $2.02 against $1.57 a year earlier). The seven needs-data gaps ask for things no tool returns, such as HomeView sales, historical P/E averages and the new CEO's background.

The field-by-field comparison shows the revisions acting on the fixable list. The summary now reads "28.7% YoY earnings growth". The net margin is given as 27.2% for Q3 FY2026 from the quarterly statement instead of an unlabeled 27.6%. The thesis drops the claim that the P/E is "elevated even by historical standards", which no gathered fact supported (the critic listed historical P/E averages as missing data). The JP Morgan upside point now also gives the consensus target, and that is where the "4.2% downside" error discussed in 7.2 entered.

7.4 Learning Across Runs

The memory layer saves the most useful lessons from each run into a ticker-specific file in the memory directory. The next run loads those notes before planning, so the agent can reuse prior findings and go after the gaps the last run left open. The results below show what the notes changed in run 2 and what they could not change.

[30]:
```
print("Notes saved after run 1, by category:")
for cat in memory.CATEGORIES:
    notes = [n["note"] for n in run1["notes_saved"]
             if n["category"] == cat]
    print(f"\n{cat} ({len(notes)})")
    for note in notes:
        print(wrap(note, indent=" - "))
print("\nFiles:", sorted(p.name for p in DEMO_MEMORY.glob("*.json")))
```

```
Notes saved after run 1, by category:

finding (2)
 - Prior view: Hold (medium conviction). Thesis: Apple's operational excellence is evident in 8
   consecutive quarterly earnings beats (averaging 4.8% surprise, latest 7.4% on July 30, 2026) and
   16.4% YoY revenue growth in Q3 FY2026 (ended June 30, 2026).
 - Prior key numbers: 1-year return: +36.6%; 52-week range: 246.03 to 343.46; market cap: $5.01T

lesson (3)
 - Earlier report gap: HomeView app compatibility issues: Report characterizes as a 'risk' but facts
   contain only news digest mention of 'app compatibility questions.' SEC filings or customer
   feedback data would strengthen this claim versus treating it as speculative.
 - Earlier report gap: iPhone Duo volume and gross margin targets: Report states '$1,999 price point
   achieves volume and gross margin targets' as a catalyst outcome, but no baseline targets or
   historical margin data provided for comparison.
 - Earlier report gap: Smartphone market saturation risks: Cited in risks section but no TAM data,
   unit sales trends, or competitive share loss data provided to substantiate.

tool (0)

Files: ['AAPL.json', 'general.json']
```

Run 1 saved five notes. The two findings record the view (Hold, medium conviction), the first sentence of the thesis and three key numbers. The three lessons are the first three needs-data gaps from the review of the kept draft, so they tell the next run what evidence was missing. There are no tool notes because no tool failed in run 1 (the simulated failure in 7.2 was not saved), so `general.json` was written empty.

[31]:
```
run2 = research_agent.run(TICKER, memory_dir=DEMO_MEMORY, save=LIVE)
print(f"Run 2 loaded {len(run2['notes_loaded'])} notes; "
      f"{run2['plan']['source']} plan, {len(run2['trace'])} tool calls "
      f"over {run2['turns']} turns, {len(run2['drafts'])} draft(s), "
      f"{run2['recommendation']}")
```

```
Run 2 loaded 5 notes; llm plan, 11 tool calls over 4 turns, 2 draft(s), Hold (medium conviction)
```

[32]:
```
print("Run 2 plan:")
display(plan_table(run2["plan"]))
print("How memory changed the plan (the planner's memory_used):")
for u in run2["plan"]["memory_used"]:
    if isinstance(u, dict):
        print(wrap(u.get("note", ""), indent=" - note:   "))
        print(wrap(u.get("effect", ""), indent="   effect: "))
    else:
        print(wrap(u, indent=" - "))
```

Run 2 plan:

	step	tools
1	Verify current price momentum and recent performance to check if prior +36.6% 1-year return and 52-week range still hold, and assess technical strength.	get_price_summary, get_price_history
2	Update company fundamentals, valuation metrics, and market cap to confirm prior $5.01T figure and assess current valuation relative to historical averages.	get_company_info
3	Validate the 8 consecutive quarterly earnings beats claim and check for any new earnings surprises since July 30, 2026; retrieve actual reported vs. estimate EPS data.	get_earnings
4	Obtain latest financial statements (income statement, balance sheet, cash flow) to verify 16.4% YoY revenue growth claim in Q3 FY2026 and establish iPhone Duo gross margin baselines and historical margin data for comparison against stated $1,999 price point targets.	get_financials
5	Search recent SEC filings and filing text for HomeView app compatibility issues and iPhone Duo volume/gross margin targets to replace news digest mentions with substantive regulatory or management guidance.	get_recent_filings, get_filing_text
6	Gather smartphone market TAM data, unit sales trends, and competitive share loss evidence by scanning recent news and earnings call commentary to substantiate smartphone market saturation risk claim.	get_news, get_news_digest
7	Assess current macroeconomic backdrop (interest rates, inflation, consumer spending, tech sector headwinds) and cross-check against any company-specific catalysts or risks in recent news to finalize Buy/Hold/Sell recommendation.	get_macro_snapshot, get_news_digest

```
How memory changed the plan (the planner's memory_used):
 - note:   Earlier report gap: HomeView app compatibility issues treated as speculative with only
           news digest mention
   effect: Added get_recent_filings and get_filing_text to Step 5 to retrieve actual SEC filings or
           management statements rather than relying solely on news digests; this provides evidence-
           based substantiation or refutation of the risk.
 - note:   Earlier report gap: iPhone Duo volume and gross margin targets lack baseline targets or
           historical margin data
   effect: Moved get_financials forward to Step 4 (earlier priority) to extract historical gross
           margin data and establish baseline comparisons before evaluating the $1,999 price point
           catalyst claim.
 - note:   Earlier report gap: Smartphone market saturation risks cited without TAM data, unit sales
           trends, or competitive share loss data
   effect: Added explicit search in Step 6 for market TAM, unit sales trends, and competitive share
           metrics via get_news and get_news_digest to substantiate or refute the saturation risk
           with quantitative evidence.
 - note:   Prior view and key numbers from earlier run (Hold, +36.6% return, $5.01T market cap, 8
           consecutive beats, 16.4% YoY revenue growth)
   effect: Added Step 1 (get_price_summary, get_price_history) and Step 2 (get_company_info) and
           Step 3 (get_earnings) to validate whether these findings still hold and whether any major
           changes in momentum, valuation, or earnings consistency have occurred.
```

Run 2's planner received the five notes and listed four effects on its plan. They show in the steps. Run 1 had no step for HomeView, iPhone Duo margins or smartphone saturation; run 2 adds step 5 (search SEC filings for HomeView and iPhone details) and step 6 (market size and share data through the news tools), and gives `get_financials` its own step to look for gross margin history. The findings changed the framing of steps 1 to 3, which now verify the prior numbers (the +36.6% return, the $5.01T market cap, the eight straight beats) instead of collecting them from scratch.

The limit is the toolset. The lessons ask for data that no tool provides, so the planner can only point existing tools at each gap, and an SEC filing is unlikely to report HomeView app compatibility.

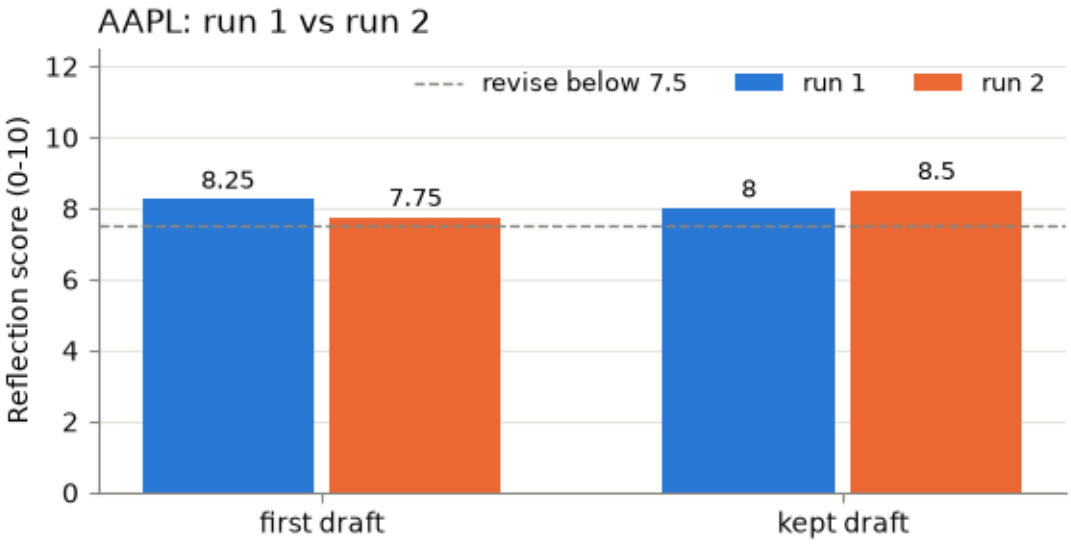
[33]:
```
def run_summary(run):
    first, best = (run["reflection_history"][0],
                   run["reflection_history"][run["best_draft"]])
    return {
        "notes loaded": len(run["notes_loaded"]),
        "plan steps": len(run["plan"]["steps"]),
        "tool calls": len(run["trace"]),
        "distinct tools": len({t["tool"] for t in run["trace"]}),
        "tool errors": sum(not t["ok"] for t in run["trace"]),
        "turns": run["turns"],
        "evidence areas": sum(run["coverage"].values()),
        "drafts": len(run["drafts"]),
        "first-draft score": first["score"],
        "kept-draft score": best["score"],
        "kept-draft grounded": (best["grounding"] or {}).get("ratio"),
        "recommendation": run["recommendation"],
    }


compare = pd.DataFrame({"run 1": run_summary(run1),
                        "run 2": run_summary(run2)})
compare
```

[33]:

	run 1	run 2
notes loaded	0	5
plan steps	7	7
tool calls	11	11
distinct tools	9	10
tool errors	0	0
turns	4	4
evidence areas	6	6
drafts	3	2
first-draft score	8.25	7.75
kept-draft score	8.0	8.5
kept-draft grounded	1.0	1.0
recommendation	Hold (medium conviction)	Hold (medium conviction)

[34]:
```
# First- and kept-draft reflection scores, run 1 vs run 2.
labels = ["first draft", "kept draft"]
fig, ax = plt.subplots(figsize=(6.5, 3))
for i, (name, run) in enumerate((("run 1", run1), ("run 2", run2))):
    vals = [compare.loc["first-draft score", name],
            compare.loc["kept-draft score", name]]
    bars = ax.bar(np.arange(2) + (i - 0.5) * 0.36, vals, 0.33,
                  color=SERIES[i], label=name)
    ax.bar_label(bars, fmt="%g", fontsize=9, padding=2)
ax.axhline(research_agent.REVISE_BELOW, color=MUTED, ls="--", lw=1,
           label=f"revise below {research_agent.REVISE_BELOW}")
ax.set(xticks=range(2), xticklabels=labels, ylim=(0, 12.5),
       ylabel="Reflection score (0-10)",
       title=f"{TICKER}: run 1 vs run 2")
ax.legend(frameon=False, ncols=3, loc="upper right", fontsize=9)
style(ax)
plt.show()
```



[35]:
```
# Run 2's reflection, and the tools it called that run 1 did not.
display(reflection_table(run2))
print("Fixable gaps on run 2's first draft:")
for g in run2["reflection_history"][0]["fixable"]:
    print(wrap(g, indent=" - "))
new_tools = sorted({t["tool"] for t in run2["trace"]}
                   - {t["tool"] for t in run1["trace"]})
print("Tools in run 2 but not in run 1:", ", ".join(new_tools) or "none")
```

draft	coverage	evidence	recency	balance	overall	grounded	ungrounded	fixable	needs data	revise
0	7	8	9	7	7.75	1.0	none	3	7	True
1	8	9	9	8	8.50	1.0	none	0	6	False

```
Fixable gaps on run 2's first draft:
 - The report states 'revenue declining 1.59% quarter-over-quarter' as a risk, but this is presented
   without context-it contradicts the bullish narrative of 16.4% YoY growth. The report should
   clarify that sequential declines are normal and focus on YoY trends instead, or explain why this
   specific QoQ decline signals weakness.
 - The report mentions 'HomeView ecosystem compatibility issues' as a risk from 'news digest,' but
   provides no quantified impact or timeline. The facts show HomeView faces 'app compatibility
   questions' but no revenue/margin impact is estimated; report should state this risk magnitude is
   unknown.
 - The report cites analyst mean target of $329.38 as 'persistent optimism,' but this is actually
   4.1% *below* the current stock price of $343.46 as of October 16, 2026. This undermines the
   bullish case and should be highlighted as a bearish signal.
Tools in run 2 but not in run 1: get_news
```

Both runs end at Hold with medium conviction, so memory changed how the agent researched rather than what it concluded. Run 2 called one tool that run 1 did not, `get_news`, for the market-share step its plan added. Its first draft scored lower than run 1's (7.75 against 8.25), and its critic found three fixable gaps. One is a real misreading: the draft called the analysts' mean target of $329.38 "persistent optimism" although it is 4.1% below the $343.46 price. A single revision fixed all three and scored 8.50, the highest score in either run.

We do not read the difference between the runs as improvement from memory alone. With one run of

each, the critic's noise is about as large as the gap between them. What memory clearly did is change the plan and the evidence gathered, as 7.4 shows.

8. End-to-End Run

`pipeline.run_all` joins the pieces. Called with only a ticker, it runs the agent, then the prompt chain with its default 7-day window, then routing and the evaluator-optimizer, and finally compares the two recommendations. Here we pass in the results from Sections 4 to 7 so the notebook does not start a third agent run with a different memory state. Run 2 stands in for the agent because it is the run that used memory.

[36]:
```
# Section 8 reuses the results above (run 2 is the agent run that had
# memory) rather than starting a third agent run with different notes.
# pipeline.run_all(TICKER) with no results passed runs every stage.
e2e = pipeline.run_all(TICKER, agent=run2, chain=chain, routed=routed,
                       loop=loop)
c = e2e["comparison"]
signals = ", ".join(f"{k}: {v}" for k, v in
                    c["specialist_signals"].items())
pd.DataFrame([
    {"stage": "Prompt chain",
     "result": chain["digest"]["headline"],
     "check": f"{len(pre['articles'])} articles kept"},
    {"stage": "Routing", "result": signals,
     "check": f"{len(routed['decisions'])} items routed"},
    {"stage": "Evaluator-optimizer", "result": f"View: {c['brief_view']}",
     "check": f"score {c['brief_score']:.3f}, passed {c['brief_passed']}"},
    {"stage": "Research agent (run 2)",
     "result": c["agent_recommendation"],
     "check": f"reflection {c['agent_reflection_score']}/10"},
]).set_index("stage")
```

[36]:

stage	result	check
Prompt chain	Apple reaches $5 trillion valuation milestone with strong demand for new foldable iPho...	18 articles kept
Routing	earnings: bullish, news: neutral, market: neutral	18 items routed
Evaluator-optimizer	View: Hold	score 0.967, passed True
Research agent (run 2)	Hold (medium conviction)	reflection 8.5/10

[37]:
```
print(f"Agent rating: {c['agent_rating']}; brief view: "
      f"{c['brief_view']}; agree: {c['agree']}")
```

```
Agent rating: Hold; brief view: Hold; agree: True
```

The agent and the evaluator-optimizer reached Hold separately, from different inputs: the agent from its own tool calls and the 10-Q, the brief from the routed specialists' views. The specialists split (earnings bullish, news and market neutral), and both final views weigh strong execution against a share price above the analysts' mean target.

[38]:
```
# LLM calls per stage, counted from each result. The chain makes one
# classify call, one extract call per batch of relevant articles, and
# one summarize call. The agent makes one planning call, one per
# tool-loop turn, one to write the report, one per reflection and one
# per revision; its news tool runs the chain again with the window the
# model chose. Routing's calls come from its saved live run.
```

```
import math


def chain_calls(result):
    relevant = sum(a["labels"].get("relevant", False)
                   for a in result["steps"]["classify"])
    return 2 + math.ceil(relevant / prompt_chain.EXTRACT_BATCH)


def agent_calls(run):
    return (1 + run["turns"] + 1 + len(run["reflection_history"])
            + len(run["drafts"]) - 1)


news_args = next(t["args"] for t in run1["trace"]
                 if t["tool"] == "get_news_digest")
agent_chain = prompt_chain.run_chain(
    TICKER, days=news_args.get("days", 7),
    limit=news_args.get("limit", 30), save=False)
n = len(loop["history"])
costs = pd.DataFrame([
    {"stage": "prompt chain (7 days)", "LLM calls": chain_calls(chain)},
    {"stage": "routing",
     "LLM calls": live_runs("routing")[-1]["usage"]["calls"]},
    {"stage": "evaluator-optimizer", "LLM calls": 1 + n + (n - 1)},
    {"stage": f"agent's news tool (chain, {news_args.get('days')} days)",
     "LLM calls": chain_calls(agent_chain)},
    {"stage": "research agent run 1", "LLM calls": agent_calls(run1)},
    {"stage": "research agent run 2", "LLM calls": agent_calls(run2)},
    {"stage": "tool-failure demo (loop only)",
     "LLM calls": failed["turns"]},
]).set_index("stage")
costs.loc["total"] = costs.sum()
costs
```

[38]:

stage	LLM calls
prompt chain (7 days)	4
routing	4
evaluator-optimizer	4
agent's news tool (chain, 30 days)	4
research agent run 1	11
research agent run 2	9
tool-failure demo (loop only)	4
total	40

Everything in this notebook takes 40 LLM calls, all to Claude Haiku 4.5. The agent is the most expensive component at 9 to 11 calls per run, because every tool-loop turn, reflection and revision is a separate call. Each of the three workflows takes 4, and the agent's news tool runs the chain once more for its 30-day window (4 calls, which both agent runs and the failure demo share through the cache). On their live runs the prompt chain used 6,817 input and 3,098 output tokens and routing used 4,281 and 1,151. An offline replay makes no calls.

9. Evaluation and Iteration

We evaluated each component on its real output and changed the code when the output was wrong. The evidence is the saved runs in `data/runs/`, one file per live run, and the offline test suite.

The prompt chain went through three development runs in late September before the final run (table below). In the first, NewsAPI returned all 19 articles and Yahoo none, and 7 of the 19 were pages from PyPI, the Python package index; 9 articles were relevant. The second run shows the overcorrection: Yahoo returned 19 articles and NewsAPI 1, and preprocessing dropped 10 that never named Apple, leaving 7 relevant. Across these runs we added a Yahoo search fallback, a preferred list of business and tech outlets for NewsAPI (with PyPI and deal sites excluded when it falls back to an open search), a round-robin merge of the two providers and the cap of four articles per outlet. The third run took 15 articles from each provider, kept 15 and found 8 relevant. The final run on 2026-10-18 used the same code and found 13 relevant articles among the 18 it kept.

[39]:
```
# Every saved prompt-chain run for the ticker, oldest first: how many
# articles came in from each provider, survived preprocessing, and
# were labeled relevant. Runs that only replayed the cache are left
# out. These are the iterations described above.
rows = []
for path in sorted((config.DATA_DIR / "runs").glob(
        f"prompt_chain_{TICKER}_*.json")):
    run = json.loads(path.read_text())
    if not run["usage"]["calls"]:
        continue
    steps = run["steps"]
    providers = pd.Series([a.get("provider") for a in steps["ingest"]])
    rows.append({
        "run": run.get("run_at", path.stem[-15:]),
        "ingested": len(steps["ingest"]),
        "newsapi": int((providers == "newsapi").sum()),
        "yahoo": int((providers == "yahoo").sum()),
        "kept": len(steps["preprocess"]["articles"]),
        "relevant": sum(a["labels"].get("relevant", False)
                        for a in steps["classify"]),
        "LLM calls": run["usage"]["calls"],
    })
pd.DataFrame(rows).set_index("run")
```

[39]:

run	ingested	newsapi	yahoo	kept	relevant	LLM calls
2026-09-30T03:42:35+00:00	19	19	0	17	9	4
2026-09-30T03:55:10+00:00	20	1	19	10	7	3
2026-09-30T03:59:29+00:00	30	15	15	15	8	3
2026-10-18T21:45:36+00:00	30	15	15	18	13	4

[40]:
```
# Every live evaluator-optimizer and agent run saved for the ticker,
# oldest first. Prompts changed between rows; the text below says how.
def saved(prefix):
    for path in sorted((config.DATA_DIR / "runs").glob(
            f"{prefix}_{TICKER}_*.json")):
        run = json.loads(path.read_text())
        if run["usage"]["calls"]:
            yield path.stem[-15:], run


print("Evaluator-optimizer runs:")
display(pd.DataFrame([{
    "run": stamp,
    "overall per iteration": ", ".join(
        f"{h['overall']:.3f}" for h in r["history"]),
    "words per iteration": ", ".join(
        str(h["words"]) for h in r["history"]),
```

```
    "passed": r["passed"],
} for stamp, r in saved("eval_opt")]).set_index("run"))

print("Research agent runs (two per notebook execution):")
display(pd.DataFrame([{
    "run": stamp,
    "notes loaded": len(r["notes_loaded"]),
    "score per draft": ", ".join(
        f"{h['score']:g}" for h in r["reflection_history"]),
    "fixable per draft": ", ".join(
        str(len(h["fixable"])) if "fixable" in h else "-"
        for h in r["reflection_history"]),
    "kept draft": r["best_draft"],
} for stamp, r in saved("agent")]).set_index("run"))

# The two errors in the first live agent run's report.
first = next(saved("agent"))[1]
text = reflection.report_text(first)
for sentence in re.split(r"(?<=\.)\s+|\n", text):
    low = sentence.lower()
    if ("1.6%" in sentence and ("year" in low or "yoy" in low)
            or "78.4x" in sentence):
        print(wrap(sentence.strip().lstrip("- "), indent=" - "))
```

```
Evaluator-optimizer runs:
```

run	overall per iteration	words per iteration	passed
20261018-144615	0.867	377	True
20261018-144718	0.867	377	True
20261018-144852	0.800, 0.767, 0.794	377, 481, 546	False
20261018-144954	0.800, 0.767, 0.797	377, 443, 511	False
20261018-145056	0.800, 0.800, 0.796	377, 482, 526	False
20261018-145245	0.767, 0.767, 0.754	377, 443, 511	False
20261018-145343	0.767, 0.967	377, 443	True

```
Research agent runs (two per notebook execution):
```

run	notes loaded	score per draft	fixable per draft	kept draft
20261018-145533	0	8.25	-	0
20261018-145618	5	8.25	-	0
20261018-150000	0	8, 8, 8	6, 4, 4	0
20261018-150146	5	7.5, 7.5, 7.5	5, 3, 5	1
20261018-150307	0	8.25, 8, 8.25	3, 4, 3	0
20261018-150448	5	7.75, 7.75, 8.25	4, 5, 3	2
20261018-150550	0	8.25, 8, 8	3, 3, 0	2
20261018-150654	5	7.75, 8.5	3, 0	1

```
 - Revenue growth remains modest at 1.6% year-over-year (June 2026 quarter), but earnings growth of
   28.7% and profitability margins of 27.6% underscore pricing power and operational leverage,
   particularly around premium products like the $1,999 foldable iPhone that sold out in minutes.
 - iPhone revenue headwinds evident in June 2026 quarter with -1.6% YoY revenue decline despite
   foldable launch; analyst concerns about core iPhone demand cited in news flow alongside JP
   Morgan's $360 target (implying caution)
 - Macroeconomic pressures with 10-year Treasury yield at 5.1% (up 105 bps YoY) and CPI inflation at
   3.72% YoY reduce consumer discretionary purchasing power; debt-to-equity of 78.4x indicates
   significant financial leverage sensitivity to rate changes
```

The tables above list every live run of the evaluator-optimizer and the agent on the day of the final run. The prompts changed between rows.

Evaluator-optimizer. In the first two rows the brief passed on its first draft at 0.867, so nothing was refined, although the grader's own feedback called the sell trigger unmeasurable: it gave 4 out of 5 to criteria it had just criticized. In row 2 we asked the grader to make its scores agree with its feedback, which changed nothing. Rows 3 to 6 add the score anchors and then instructions for the optimizer. Now no draft passes in three iterations, scores stay between 0.754 and 0.800, and revisions grow from 377 to as many as 546 words. Each review raised new optional points and the optimizer added text to answer them. Telling the optimizer to stay within the word limit did not stop the growth (row 6). What worked was giving the grader its previous feedback and limiting new points to errors and missing required elements (row 7): the first draft fails at 0.767 and the revision passes at 0.967.

Self-reflection. In the first live run (the first two agent rows) every draft scored 8.25 and none was revised, but the report contained the two errors quoted under the tables. It called a quarter-over-quarter revenue change of 1.6% a year-over-year figure, and it gave debt-to-equity as 78.4x when Yahoo reports that ratio in percent (0.78x). The critic missed both. We fixed the inputs, so the facts now name the period of each growth rate and the unit of debt-to-equity, and we asked the critic to check periods and units and to split its gaps into those the facts can fix and those that need new data. Rows 3 and 4 show the next problem: every draft was revised twice with flat scores, because the critic filed gaps that needed new data as fixable. Requiring each fixable gap to name its fact (rows 5 and 6) still left three to five fixable gaps on every draft, as the critic found new small items each round. Giving the critic its previous list on a re-review, the same fix as for the evaluator, let both runs settle (rows 7 and 8, the runs shown in Section 7).

[41]:
```
# The offline test suite (mocked LLM and APIs), run from the repo root.
result = subprocess.run(
    [sys.executable, "-m", "pytest", "-q", "-p", "no:cacheprovider",
     "--color=no"],
    cwd=config.ROOT, capture_output=True, text=True)
summary = result.stdout.strip().splitlines()[-1]
print(re.sub(r" in [\d.]+s", "", summary))
```

```
97 passed
```

All 97 tests in `tests/` pass. They run offline: the LLM and every API are replaced by fakes, so they test our code's logic (parsing, grounding, routing rules, when the loops stop, memory files, caching and offline replay) rather than the model's judgment. They run with `ruff` before every commit.

The largest known limitation is in the grounding checks, which ask whether a number appears anywhere in the facts. They accept a number that belongs to a different fact (the agent's "4.2% downside" passed because unemployment is 4.2%) and reject correct arithmetic (the 3.5-point P/E gap). Matching each number to the field it claims to come from would fix both, at the cost of parsing what each sentence refers to.

Both critics are Claude Haiku 4.5 judging Claude Haiku 4.5's writing. Scores move by a quarter point between drafts without a material change, and neither grader caught the brief's "YTD return" label or its length. A stronger grading model, or a few runs per draft, would make the scores more trustworthy. Self-reflection and memory can also only ask for data our tools provide, so lessons such as smartphone market size stay open from run to run.

Two smaller issues remain. The agent's last tool-loop turn writes a full report instead of the requested two or three sentences; the report we show comes from a separate writer call, so this costs output tokens but not accuracy. And everything here is one ticker on one day, so we did not test whether the recommendations would have been right.

10. AI Tool Disclosure

We used AI tools to build this project, and the system itself is built on an LLM.

The system calls Anthropic's Claude API (Claude Haiku 4.5, model `claude-haiku-4-5`) for every LLM step: the prompt chain's classify, extract and summarize steps, the router and the three specialists, the generator, evaluator and optimizer, and the agent's planner, tool loop, report writer and critic. Every model reply in this notebook is that model's output, cached and replayed as described in Section 2.

The team used Claude (claude.ai and Claude Code, both from Anthropic) across the workstreams (data tools, prompt chain, routing and evaluator-optimizer, research agent) and for the final consolidation: the fixes that make all four agent functions run, offline replay, the evaluator and reflection changes described in Section 9, the notebook code, and first drafts of the notebook's prose.

Citations: Anthropic, Claude Haiku 4.5 (`claude-haiku-4-5`) through the Claude API (https://platform.claude.com), called by the system for the live run on 2026-10-18 shown here. Anthropic, Claude (https://claude.ai) and Claude Code, used by the team during development in September and October 2026.

We checked AI-written code by reading it, with the offline test suite, and by running it live and inspecting the outputs shown here; several of the changes in Section 9 came from that inspection. Every number in the prose was checked against the executed output.